

++ and --

- (++) increments a pointer to point to the next array element

```
int_ptr++;
```

- (--) decrements a pointer to point to the previous array element

```
int_ptr--;
```

+ and -

- (+) increment pointer by $n * \text{sizeof}(\text{type})$

`int_ptr += n; or int_ptr = int_ptr + n;`

- (-) decrement pointer by $n * \text{sizeof}(\text{type})$

`int_ptr -= n; or int_ptr = int_ptr - n;`

Subtracting two pointers

- Determine the number of elements between the pointers
- Both pointers must point to the same data type

```
int n = int_ptr2 - int_ptr1;
```

Comparing two pointers == and !=

Determine if two pointers point to the same location

- does NOT compare the data where they point!

```
string s1 {"Frank"};  
string s2 {"Frank"};
```

```
string *p1 {&s1};  
string *p2 {&s2};  
string *p3 {&s1};
```

```
cout << (p1 == p2) << endl;    // false  
cout << (p1 == p3) << endl;    // true
```


Comparing the data pointers point to

Determine if two pointers point to the same data

- you must compare the referenced pointers

```
string s1 {"Frank"};  
string s2 {"Frank"};
```

```
string *p1 {&s1};  
string *p2 {&s2};  
string *p3 {&s1};
```

```
cout << (*p1 == *p2) << endl;    // true  
cout << (*p1 == *p3) << endl;    // true
```

Passing pointers to a function

`const` and Pointers

- There are several ways to qualify pointers using `const`
 - Pointers to constants
 - Constant pointers
 - Constant pointers to constants

Pointers to constants

- The data pointed to by the pointers is constant and **cannot** be changed.
- The pointer itself can change and point somewhere else.

```
int high_score {100};  
int low_score { 65};  
const int *score_ptr { &high_score };
```

```
*score_ptr = 86;    // ERROR  
score_ptr = &low_score; // OK
```

Constant pointers

- The data pointed to by the pointers can be changed.
- The pointer itself **cannot** change and point somewhere else

```
int high_score {100};  
int low_score { 65};  
int *const score_ptr { &high_score };
```

```
*score_ptr = 86;    // OK  
score_ptr = &low_score; // ERROR
```

Constant pointers to constants

- The data pointed to by the pointer is constant and **cannot** be changed.
- The pointer itself **cannot** change and point somewhere else.

```
int high_score {100};  
int low_score { 65};  
const int *const score_ptr { &high_score };
```

```
*score_ptr = 86;    // ERROR  
score_ptr = &low_score; // ERROR
```


Passing pointers to a function

- Pass-by-reference with pointer parameters
- We can use pointers and the dereference operator to achieve pass-by-reference
- The function parameter is a pointer
- The actual parameter can be a pointer or address of a variable

Passing pointers to a function

Pass-by-reference with pointers – defining the function

```
void double_data(int *int_ptr) ;

void double_data(int *int_ptr) {
    *int_ptr *= 2;

    // *int_ptr = *int_ptr * 2;
}
```

Passing pointers to a function

Pass-by-reference with pointers – calling the function

```
int main() {  
    int value {10};  
  
    cout << value << endl;    // 10  
  
    double_data( &value );  
  
    cout << value << endl;    // 20  
}
```


Returning a Pointer from a Function

- Functions can also return pointers

```
type *function();
```

- Should return pointers to
 - Memory dynamically allocated in the function
 - To data that was passed in
- Never return a pointer to a local function variable!

returning a parameter

```
int *largest_int(int *int_ptr1, int *int_ptr2) {  
    if (*int_ptr1 > *int_ptr2)  
        return int_ptr1;  
    else  
        return int_ptr2;  
}
```

returning a parameter

```
int main() {  
    int a{100};  
    int b{200};  
  
    int *largest_ptr {nullptr};  
    largest_ptr = largest_int(&a, &b);  
    cout << *largest_ptr << endl;    // 200  
    return 0;  
}
```

returning dynamically allocated memory

```
int *create_array(size_t size, int init_value = 0) {  
  
    int *new_storage {nullptr};  
  
    new_storage = new int[size];  
    for (size_t i{0}; i < size; ++i)  
        *(new_storage + i) = init_value;  
    return new_storage;  
}
```

returning dynamically allocated memory

```
int main() {  
    int *my_array;           // will be allocated by the function  
  
    my_array = create_array(100,20); // create the array  
  
    // use it  
  
    delete [] my_array; /      / be sure to free the storage  
    return 0;  
}
```


Never return a pointer to a local variable!!

```
int *dont_do_this () {  
    int size {};  
    . . .  
    return &size;  
}  
  
int *or_this () {  
    int size {};  
    int *int_ptr {&size};  
    . . .  
    return int_ptr;  
}
```

Potential Pointer Pitfalls

- Uninitialized pointers
- Dangling Pointers
- Not checking if new failed to allocate memory
- Leaking memory

Uninitialized pointers

```
int *int_ptr; // pointing anywhere
```

```
. . .
```

```
*int_ptr = 100; // Hopefully a crash
```


Dangling pointer

- Pointer that is pointing to released memory
 - For example, 2 pointers point to the same data
 - 1 pointer releases the data with delete
 - The other pointer accesses the release data
- Pointer that points to memory that is invalid
 - We saw this when we returned a pointer to a function local variable

Not checking if new failed

- If `new` fails an exception is thrown
- We can use exception handling to catch exceptions
- Dereferencing a null pointer will cause your program to crash

Leaking memory

- Forgetting to release allocated memory with delete
- If you lose your pointer to the storage allocated on the heap you have no way to get to that storage again
- The memory is orphaned or leaked
- One of the most common pointer problems

What is a Reference?

- An alias for a variable
- Must be initialized to a variable when declared
- Cannot be null
- Once initialized cannot be made to refer to a different variable
- Very useful as function parameters
- Might be helpful to think of a reference as a constant pointer that is automatically dereferenced

What is a reference?

Using references in range-based for loop

```
vector<string> stooges {"Larry", "Moe", "Curly"};
```

```
for (auto str: stooges)
    str = "Funny";        // changes the copy
```

```
for (auto str:stooges)
    cout << str << endl;    // Larry, Moe, Curly
```


What is a reference?

Using references in range-based for loop

```
vector<string> stooges {"Larry", "Moe", "Curly"};
```

```
for (auto &str: stooges)
    str = "Funny";    // changes the actual
```

```
for (auto str:stooges)
    cout << str << endl;    // Funny, Funny, Funny
```

What is a reference?

Using references in range-based for loop

```
vector<string> stooges {"Larry", "Moe", "Curly"};
```

```
for (auto const &str: stooges)  
    str = "Funny";    // compiler error
```

What is a reference?

Using references in range-based for loop

```
vector<string> stooges {"Larry", "Moe", "Curly"};
```

```
for (auto const &str:stooges)  
    cout << str << endl;    // Larry, Moe, Curly
```


What is a reference?

Passing references to functions

- Please refer to the section 11 videos and examples

l-values

- l-values
 - values that have names and are addressable
 - modifiable if they are not constants

```
int x {100};    // x is an l-value
```

```
x = 1000;
```

```
x = 1000 + 20;
```

```
string name;    // name is an l-value
```

```
name = "Frank";
```

l-values

- l-values
 - values that have names and are addressable
 - modifiable if they are not constants

```
100 = x;           // 100 is NOT an l-value
```

```
(1000 + 20) = x;  // (1000 + 20) is NOT an l-value
```

```
string name;
```

```
name = "Frank";
```

```
"Frank" = name;  // "Frank" is NOT an l-value
```

r-values

- r-value (non-addressable and non-assignable)
 - A value that's not an l-value
 - on the right-hand side of an assignment expression
 - a literal
 - a temporary which is intended to be non-modifiable

```
int x {100};           // 100 is an r-value  
int y = x + 200;       // (x+200) is an r-value
```

```
string name;  
name = "Frank";       // "Frank" is an r-value
```

```
int max_num = max(20,30); // max(20,30) is an r-value
```

r-values

- r-values can be assigned to l-values explicitly

```
int x {100};  
int y {0};
```

```
y = 100;    // r-value 100 assigned to l-value y
```

```
x = x + y;  // r-value (x+y) assigned to l-value x
```


I-value references

- The references we've used are l-value references
 - Because we are referencing l-values

```
int x {100};
```

```
int &ref1 = x;      // ref1 is reference to l-value  
ref1 = 1000;
```

```
int &ref2 = 100; // Error 100 is an r-value
```

l-value references

- The same when we pass-by-reference

```
int square(int &n) {  
    return n*n;  
}
```

```
int num {10};
```

```
square(num) ;    // OK
```

```
square(5) ;    // Error - can't reference r-value 5
```

When to use pointers vs. references parameters

- Pass-by-value
 - when the function does **not** modify the actual parameter, and
 - the parameter is small and efficient to copy like simple types (int, char, double, etc.)

When to use pointers vs. references parameters

- Pass-by-reference using a pointer
 - when the function does modify the actual parameter, and
 - the parameter is expensive to copy, and
 - Its OK to the pointer is allowed a nullptr value

When to use pointers vs. references parameters

- Pass-by-reference using a pointer to `const`
 - when the function does **not** modify the actual parameter, and
 - the parameter is expensive to copy, and
 - Its OK to the pointer is allowed a `nullptr` value

When to use pointers vs. references parameters

- Pass-by-reference using a const pointer to const
 - when the function does **not** modify the actual parameter, and
 - the parameter is expensive to copy, and
 - Its OK to the pointer is allowed a nullptr value, and
 - You don't want to modify the pointer itself

When to use pointers vs. references parameters

- Pass-by-reference using a reference
 - when the function **does** modify the actual parameter, and
 - the parameter is expensive to copy, and
 - The parameter will never be nullptr

When to use pointers vs. references parameters

- Pass-by-reference using a `const` reference
 - when the function does **not** modify the actual parameter, and
 - the parameter is expensive to copy, and
 - The parameter will never be `nullptr`

Object-Oriented Programming – Classes and Objects

- What is Object-Oriented Programming?
- What are Classes and Objects?
- Declaring Classes and creating Objects
- Dot and pointer operators
- `public` and `private` access modifiers
- Methods, Constructors and Destructors
 - `class` methods
 - default and overloaded constructors
 - copy and move constructors
 - shallow vs. deep copying
 - `this` pointer
- `static` class members
- `struct` vs. `class`
- `friend` of a class

What is Object-Oriented Programming?

- Procedural Programming
- Procedural Programming limitations
- OO Programming concepts and their advantages
- OO Programming limitations

Procedural programming

- Focus is on processes or actions that a program takes
- Programs are typically a collection of functions
- Data is declared separately
- Data is passed as arguments into functions
- Fairly easy to learn

Procedural programming - Limitations

- Functions need to know the structure of the data.
 - if the structure of the data changes many functions must be changed
- As programs get larger they become more:
 - difficult to understand
 - difficult to maintain
 - difficult to extend
 - difficult to debug
 - difficult to reuse code
 - fragile and easier to break

What is Object-Oriented Programming?

- Classes and Objects
 - focus is on classes that model real-world domain entities
 - allows developers to think at a higher level of abstraction
 - used successfully in very large programs

What is Object-Oriented Programming?

- Encapsulation
 - objects contain data AND operations that work on that data
 - Abstract Data Type (ADT)

What is Object-Oriented Programming?

- Information-hiding
 - implementation-specific logic can be hidden
 - users of the class code to the interface since they don't need to know the implementation
 - more abstraction
 - easier to test, debug, maintain and extend

What is Object-Oriented Programming?

- Reusability
 - easier to reuse classes in other applications
 - faster development
 - higher quality

What is Object-Oriented Programming?

- Inheritance
 - can create new classes in term of existing classes
 - reusability
 - polymorphic classes
- Polymorphism and more...

Limitations

- Not a panacea
 - OO Programming won't make bad code better
 - not suitable for all types of problems
 - not everything decomposes to a class
- Learning curve
 - usually a steeper learning curve, especially for C++
 - many OO languages, many variations of OO concepts
- Design
 - usually more up-front design is necessary to create good models and hierarchies
- Programs can be:
 - larger in size
 - slower
 - more complex

Classes and Objects

- **Classes**

- blueprint from which objects are created
- a user-defined data-type
- has attributes (data)
- has methods (functions)
- can hide data and methods
- provides a public interface

- **Example classes**

- Account
- Employee
- Image
- `std::vector`
- `std::string`

Classes and Objects

- **Objects**

- created from a class
- represents a specific instance of a class
- can create many, many objects
- each has its own identity
- each can use the defined class methods

- **Example Account objects**

- Frank's account is an instance of an Account
- Jim's account is an instance of an Account
- Each has its own balance, can make deposits, withdrawals, etc.

Classes and Objects

```
int high_score;  
int low_score;
```

```
Account frank_account;  
Account jim_account;
```

```
std::vector<int> scores;  
std::string name;
```

Declaring a Class

```
class Class_Name  
{  
  
    // declaration(s);  
  
};
```

Player

```
class Player
{
    // attributes
    std::string name;
    int health;
    int xp;

    // methods
    void talk(std::string text_to_say);
    bool is_dead();
};
```

Creating objects

```
Player frank;
```

```
Player hero;
```

```
Player *enemy = new Player();
```

```
delete enemy;
```


Account

```
class Account
{
    // attributes
    std::string name;
    double balance;

    // methods
    bool withdraw(double amount);
    bool deposit(double amount);
};
```

Creating objects

```
Account frank_account;  
Account jim_account;
```

```
Account *mary_account = new Account();  
delete mary_account;
```

Creating objects

```
Account frank_account;
```

```
Account jim_account;
```

```
Account accounts[] {frank_account, jim_account};
```

```
std::vector<Account> accounts1 {frank_account};  
accounts1.push_back(jim_account);
```


Accessing Class Members

- We can access
 - class attributes
 - class methods
- Some class members will not be accessible (more on that later)
- We need an object to access instance variables

Accessing Class Members

If we have an object (dot operator)

- **Using the dot operator**

```
Account frank_account;
```

```
frank_account.balance;
```

```
frank_account.deposit(1000.00);
```

Accessing Class Members

If we have a pointer to an object (member of pointer operator)

- Dereference the pointer then use the dot operator.

```
Account *frank_account = new Account();
```

```
(*frank_account).balance;
```

```
(*frank_account).deposit(1000.00);
```

- Or use the member of pointer operator (arrow operator)

```
Account *frank_account = new Account();
```

```
frank_account->balance;
```

```
frank_account->deposit(1000.00);
```

Class Member Access Modifiers

`public`, `private`, and `protected`

- `public`
 - accessible everywhere
- `private`
 - accessible only by members or friends of the class
- `protected`
 - used with inheritance – we'll talk about it in the next section

Class Member Access Modifiers

`public`

```
class Class_Name
{
public:

    // declaration(s);

};
```


Class Member Access Modifiers

private

```
class Class_Name
{
private:

    // declaration(s);

};
```

Class Member Access Modifiers

protected

```
class Class_Name
{
protected:

    // declaration(s);

};
```

Declaring a Class

Player

```
class Player
{
private:
    std::string name;
    int health;
    int xp;
public:
    void talk(std::string text_to_say);
    bool is_dead();
};
```


Creating objects

```
Player frank;  
frank.name = "Frank";           // Compiler error  
frank.health = 1000;            // Compiler error  
frank.talk("Ready to battle"); // OK
```

```
Player *enemy = new Player();  
enemy->xp = 100;                // Compiler error  
enemy->talk("I will hunt you down"); // OK
```

```
delete enemy;
```

Declaring a Class

Account

```
class Account
{
private:
    std::string name;
    double balance;

public:
    bool withdraw(double amount);
    bool deposit(double amount);
};
```

Creating objects

```
Account frank_account;  
frank_account.balance = 10000000.00; // Compiler error  
frank_account.deposit(10000000.0);    // OK  
frank_account.name = "Frank's Account"; // Compiler error
```

```
Account *mary_account = new Account();
```

```
mary_account->balance = 10000.0; // Compiler error  
mary_account->withdraw(10000.0); // OK
```

```
delete mary_account;
```

Implementing Member Methods

- Very similar to how we implemented functions
- Member methods have access to member attributes
 - So you don't need to pass them as arguments!
- Can be implemented inside the class declaration
 - Implicitly inline
- Can be implemented outside the class declaration
 - Need to use `Class_name::method_name`
- Can separate specification from implementation
 - .h file for the class declaration
 - .cpp file for the class implementation

Implementing Member Methods

Inside the class declaration

```
class Account {  
  
private:  
    double balance;  
public:  
    void set_balance(double bal) {  
        balance = bal;  
    }  
    double get_balance() {  
        return balance;  
    }  
  
};
```


Implementing Member Methods

Outside the class declaration

```
class Account {  
  
private:  
    double balance;  
public:  
    void set_balance(double bal);  
    double get_balance();  
};  
  
void Account::set_balance(double bal) {  
    balance = bal;  
}  
double Account::get_balance() {  
    return balance;  
}
```

Separating Specification from Implementation

Account.h

```
class Account {  
  
private:  
    double balance;  
public:  
    void set_balance(double bal);  
    double get_balance();  
};
```

Separating Specification from Implementation

Include Guards

```
#ifndef _ACCOUNT_H_  
#define _ACCOUNT_H_
```

```
    // Account class declaration
```

```
#endif
```


Separating Specification from Implementation

Account.h

```
#ifndef _ACCOUNT_H_
```

```
#define _ACCOUNT_H_
```

```
class Account {
```

```
private:
```

```
    double balance;
```

```
public:
```

```
    void set_balance(double bal);
```

```
    double get_balance();
```

```
};
```

```
#endif
```

Separating Specification from Implementation

Account.h - #pragma once

#pragma once

```
class Account {  
  
private:  
    double balance;  
public:  
    void set_balance(double bal);  
    double get_balance();  
};
```

Separating Specification from Implementation

Account.cpp

```
#include "Account.h"
```

```
void Account::set_balance(double bal) {  
    balance = bal;  
}
```

```
double Account::get_balance() {  
    return balance;  
}
```

Separating Specification from Implementation

main.cpp

```
#include <iostream>
```

```
#include "Account.h"
```

```
int main() {
```

```
    Account frank_account;
```

```
    frank_account.set_balance(1000.00);
```

```
    double bal = frank_account.get_balance();
```

```
    std::cout << bal << std::endl; // 1000
```

```
    return 0;
```

```
}
```

Constructors

- Special member method
- Invoked during object creation
- Useful for initialization
- Same name as the class
- No return type is specified
- Can be overloaded

Player Constructors

```
class Player
{
private:
    std::string name;
    int health;
    int xp;
public:
    // Overloaded Constructors
    Player();
    Player(std::string name);
    Player(std::string name, int health, int xp);
};
```

Account Constructors

```
class Account
{
private:
    std::string name;
    double balance;
public:
    // Constructors
    Account() ;
    Account(std::string name, double balance);
    Account(std::string name);
    Account(double balance);
};
```

Destructors

- Special member method
- Same name as the class preceded with a tilde (~)
- Invoked automatically when an object is destroyed
- No return type and no parameters
- Only 1 destructor is allowed per class – cannot be overloaded!
- Useful to release memory and other resources

Player Destructor

```
class Player
{
private:
    std::string name;
    int health;
    int xp;
public:
    Player();
    Player(std::string name);
    Player(std::string name, int health, int xp);
    // Destructor
    ~Player();
};
```

Account Destructor

```
class Account
{
private:
    std::string name;
    double balance;
public:
    Account();
    Account(std::string name, double balance);
    Account(std::string name);
    Account(double balance);
    // Destructor
    ~Account();
};
```

Creating objects

```
{  
    Player slayer;  
    Player frank {"Frank", 100, 4 };  
    Player hero {"Hero"};  
    Player villain {"Villain"};  
    // use the objects  
} // 4 destructors called
```

```
Player *enemy = new Player("Enemy", 1000, 0);  
delete enemy; // destructor called
```

The Default Constructor

- Does not expect any arguments
 - Also called the no-args constructor
- If you write no constructors at all for a class
 - C++ will generate a Default Constructor that does nothing
- Called when you instantiate a new object with no arguments

```
Player frank;  
Player *enemy = new Player;
```

Declaring a Class

Account – using default constructor

```
class Account
{
private:
    std::string name;
    double balance;
public:
    bool withdraw(double amount);
    bool deposit(double amount);
};
```


Creating objects

Using the default constructor

```
Account frank_account;
```

```
Account jim_account;
```

```
Account *mary_account = new Account;  
delete mary_account;
```

Declaring a Class

Account – user-defined no-args constructor

```
class Account
{
private:
    std::string name;
    double balance;
public:
    Account() {
        name = "None";
        balance = 0.0;
    }
    bool withdraw(double amount);
    bool deposit(double amount);
};
```

Declaring a Class

Account – no default constructor

```
class Account
{
private:
    std::string name;
    double balance;
public:
    Account(std::string name_val, double bal) {
        name = name_val;
        balance = bal;
    }
    bool withdraw(double amount);
    bool deposit(double amount);
};
```


Creating objects

Using the default constructor

```
Account frank_account;           // Error
Account jim_account;            // Error

Account *mary_account = new Account; // Error
delete mary_account;

Account bill_account {"Bill", 15000.0}; // OK
```

Overloading Constructors

- Classes can have as many constructors as necessary
- Each must have a unique signature
- Default constructor is no longer compiler-generated once another constructor is declared

Constructors and Destructors

Overloaded Constructors

```
class Player
{
private:
    std::string name;
    int health;
    int xp;
public:
    // Overloaded Constructors
    Player();
    Player(std::string name_val);
    Player(std::string name_val, int health_val, int xp_val);
};
```

Constructors and Destructors

Overloaded Constructors

```
Player::Player() {  
    name = "None";  
    health = 0;  
    xp = 0;  
}
```

```
Player::Player(std::string name_val) {  
    name = name_val;  
    health = 0;  
    xp = 0;  
}
```

Constructors and Destructors

Overloaded Constructors

```
Player::Player(std::string name_val, int health_val, int  
xp_val) {  
    name = name_val;  
    health = health_val;  
    xp = xp_val;  
}
```


Creating objects

```
Player empty; // None, 0, 0

Player hero {"Hero"}; // Hero, 0, 0
Player villain {"Villain"}; // Villain, 0, 0

Player frank {"Frank", 100, 4}; // Frank, 100, 4

Player *enemy = new Player("Enemy", 1000, 0); // Enemy, 1000, 0
delete enemy;
```

Constructor Initialization Lists

- So far, all data member values have been set in the constructor body
- Constructor initialization lists
 - are more efficient
 - initialization list immediately follows the parameter list
 - initializes the data members as the object is created!
 - order of initialization is the order of declaration in the class

Constructor Initialization Lists

```
class Player
{
private:
    std::string name;
    int health;
    int xp;
public:
    // Overloaded Constructors
    Player();
    Player(std::string name_val);
    Player(std::string name_val, int health_val, int xp_val);
};
```


Constructor Initialization Lists

Player()

Previous way:

```
Player::Player() {  
    name = "None";    // assignment not initialization  
    health = 0;  
    xp = 0;  
}
```

Better way:

```
Player::Player()  
    : name{"None"}, health{0}, xp{0} {  
}
```

Constructor Initialization Lists

Player(std::string)

Previous way:

```
Player::Player(std::string name_val) {  
    name = name_val;    // assignment not initialization  
    health = 0;  
    xp = 0;  
}
```

Better way:

```
Player::Player(std::string name_val)  
    : name{name_val}, health{0}, xp{0} {  
}
```

Constructor Initialization Lists

Player(std::string, int, int)

Previous way:

```
Player::Player(std::string name_val, int health_val, int xp_val) {  
    name = name_val;           // assignment not initialization  
    health = health_val;  
    xp = xp_val;  
}
```

Better way:

```
Player::Player(std::string name_val, int health_val, int xp_val)  
    : name{name_val}, health{health_val}, xp{xp_val} {  
}
```

Constructor Initialization Lists

```
Player::Player()  
    : name{"None"}, health{0}, xp{0} {  
}
```

```
Player::Player(std::string name_val)  
    : name{name_val}, health{0}, xp{0} {  
}
```

```
Player::Player(std::string name_val, int health_val, int  
xp_val)  
    : name{name_val}, health{health_val}, xp{xp_val} {  
}
```

Delegating Constructors

- Often the code for constructors is very similar
- Duplicated code can lead to errors
- C++ allows delegating constructors
 - code for one constructor can call another in the initialization list
 - avoids duplicating code

Delegating Constructors

```
class Player
{
private:
    std::string name;
    int health;
    int xp;
public:
    // Overloaded Constructors
    Player();
    Player(std::string name_val);
    Player(std::string name_val, int health_val, int xp_val);
};
```

Delegating Constructors

```
Player::Player()  
    : name{"None"}, health{0}, xp{0} {  
}
```

```
Player::Player(std::string name_val)  
    : name{name_val}, health{0}, xp{0} {  
}
```

```
Player::Player(std::string name_val, int health_val, int  
xp_val)  
    : name{name_val}, health{health_val}, xp{xp_val} {  
}
```

Delegating Constructors

```
Player::Player(std::string name_val, int health_val, int xp_val)
    : name{name_val}, health{health_val}, xp{xp_val} {
}
```

```
Player::Player()
    : Player {"None", 0, 0} {
}
```

```
Player::Player(std::string name_val)
    : Player { name_val, 0, 0} {
}
```


Default Constructor Parameters

- Can often simplify our code and reduce the number of overloaded constructors
- Same rules apply as we learned with non-member functions

Default Constructor Parameters

```
class Player
{
private:
    std::string name;
    int health;
    int xp;
public:
    // Constructor with default parameter values
    Player(std::string name_val = "None",
          int health_val = 0,
          int xp_val = 0);
};
```

Default Constructor Parameters

```
Player::Player(std::string name_val, int health_val, int
xp_val)
    : name {name_val}, health {health_val}, xp {xp_val} {

}

Player empty; // None, 0, 0
Player frank {"Frank"}; // Frank, 0, 0
Player villain {"Villain", 100, 55}; // Villain, 100, 55
Player hero {"Hero", 100}; // Hero, 100, 0

// Note what happens if you declare a no-args constructor
```

Copy Constructor

- When objects are copied C++ must create a new object from an existing object
- When is a copy of an object made?
 - passing object by value as a parameter
 - returning an object from a function by value
 - constructing one object based on another of the same class
- C++ must have a way of accomplishing this so it provides a compiler-defined copy constructor if you don't

Pass object by-value

```
Player hero {"Hero", 100, 20};

void display_player(Player p) {
    // p is a COPY of hero in this example
    // use p
    // Destructor for p will be called
}

display_player(hero);
```


Return object by value

```
Player enemy;
```

```
Player create_super_enemy() {  
    Player an_enemy{"Super Enemy", 1000, 1000};  
    return an_enemy;    // A COPY of an_enemy is returned  
}
```

```
enemy = create_super_enemy();
```

Construct one object based on another

```
Player hero {"Hero", 100, 100};
```

```
Player another_hero {hero}; // A COPY of hero is made
```

Copy Constructor

- If you don't provide your own way of copying objects by value then the compiler provides a default way of copying objects
- Copies the values of each data member to the new object
 - default memberwise copy
- Perfectly fine in many cases
- Beware if you have a pointer data member
 - Pointer will be copied
 - Not what it is pointing to
 - Shallow vs. Deep copy – more in the next video

Best practices

- Provide a copy constructor when your class has raw pointer members
- Provide the copy constructor with a **const reference** parameter
- Use STL classes as they already provide copy constructors
- Avoid using raw pointer data members if possible

Declaring the Copy Constructor

```
Type::Type(const Type &source);
```

```
Player::Player(const Player &source);
```

```
Account::Account(const Account &source);
```

Implementing the Copy Constructor

```
Type::Type(const Type &source) {  
    // code or initialization list to copy the object  
}
```

Implementing the Copy Constructor

Player

```
Player::Player(const Player &source)  
    : name{source.name},  
      health {source.health},  
      xp {source.xp} {  
  
}
```

Implementing the Copy Constructor

Account

```
Account::Account(const Account &source)  
    : name{source.name},  
      balance {source.balance} {  
  
}
```


Shallow vs. Deep Copying

- Consider a class that contains a pointer as a data member
- Constructor allocates storage dynamically and initializes the pointer
- Destructor releases the memory allocated by the constructor
- What happens in the default copy constructor?

Default copy constructor

- memberwise copy
- Each data member is copied from the source object
- The pointer is copied NOT what it points to (shallow copy)
- **Problem:** when we release the storage in the destructor, the other object still refers to the released storage!

Copy Constructor

Shallow

```
class Shallow {  
private:  
    int *data;                                // POINTER  
public:  
    Shallow(int d);                            // Constructor  
    Shallow(const Shallow &source);            // Copy  
  
    Constructor  
    ~Shallow();                                // Destructor  
};
```


Copy Constructor

Shallow constructor

```
Shallow::Shallow(int d) {  
    data = new int;    // allocate storage  
    *data = d;  
}
```

Copy Constructor

Shallow destructor

```
Shallow::~~Shallow() {  
    delete data;    // free storage  
    cout << "Destructor freeing data" << endl;  
}
```

Copy Constructor

Shallow copy constructor

```
Shallow::Shallow(const Shallow &source)
    : data(source.data) {
    cout << "Copy constructor - shallow"
        << endl;
}
```

Shallow copy - only the pointer is copied - not what it is pointing to!

Problem: `source` and the newly created object BOTH point to the SAME data area!

Copy Constructor

Shallow – a simple method that expects a copy

```
Shallow::Shallow(const Shallow &source)
: data(source.data) {
    cout << "Copy constructor - shallow"
        << endl;
}
```

Shallow copy - only the pointer is copied - not what it is pointing to!

Problem: source and the newly created object BOTH point to the SAME data area!

Copy Constructor

Sample main - will likely crash

```
int main() {  
    Shallow obj1 {100};  
    display_shallow(obj1);  
    // obj1's data has been released!  
  
    obj1.set_data_value(1000);  
    Shallow obj2 {obj1};  
    cout << "Hello world" << endl;  
    return 0;  
}
```


User-provided copy constructor

- Create a **copy** of the pointed-to data
- Each copy will have a pointer to unique storage in the heap
- Deep copy when you have a raw pointer as a class data member

Copy Constructor

Deep

```
class Deep {  
private:  
    int *data;           // POINTER  
public:  
    Deep(int d);         // Constructor  
    Deep(const Deep &source); // Copy Constructor  
    ~Deep();             // Destructor  
};
```

Copy Constructor

Deep constructor

```
Deep::Deep(int d) {  
    data = new int;    // allocate storage  
    *data = d;  
}
```


Copy Constructor

Deep constructor

```
Deep::~~Deep() {  
    delete data;    // free storage  
    cout << "Destructor freeing data" << endl;  
}
```

Copy Constructor

Deep constructor

```
Deep::Deep(const Deep &source)
{
    data = new int;    // allocate storage
    *data = *source.data;
    cout << "Copy constructor - deep"
          << endl;
}
```

Deep copy – create new storage and copy values

Copy Constructor

Deep copy constructor – delegating constructor

```
Deep::Deep(const Deep &source)
: Deep{*source.data} {
    cout << "Copy constructor – deep"
         << endl;
}
```

Delegate to `Deep(int)` and pass in the `int (*source.data)` source is pointing to

Copy Constructor

Deep – a simple method that expects a copy

```
void display_deep(Deep s) {  
    cout << s.get_data_value() << endl;  
}
```

When `s` goes out of scope the destructor is called and releases `data`.

No Problem: since the storage being releases is unique to `s`

Copy Constructor

Sample main – will not crash

```
int main() {  
    Deep obj1 {100};  
    display_deep(obj1);  
  
    obj1.set_data_value(1000);  
    Deep obj2 {obj1};  
  
    return 0;  
}
```


Move Constructor

- Sometimes when we execute code the compiler creates unnamed temporary values

```
int total {0};  
total = 100 + 200;
```

- `100 + 200` is evaluated and `300` stored in an unnamed temp value
 - the `300` is then stored in the variable `total`
 - then the temp value is discarded
-
- The same happens with objects as well

When is it useful?

- Sometimes copy constructors are called many times automatically due to the copy semantics of C++
- Copy constructors doing deep copying can have a significant performance bottleneck
- C++11 introduced move semantics and the move constructor
- Move constructor moves an object rather than copy it
- Optional but recommended when you have a raw pointer
- Copy elision – C++ may optimize copying away completely (RVO-Return Value Optimization)

r-value references

- Used in moving semantics and perfect forwarding
- Move semantics is all about r-value references
- Used by move constructor and move assignment operator to efficiently move an object rather than copy it
- R-value reference operator (&&)

r-value references

```
int x {100}
    int &l_ref = x;    // l-value reference
    l_ref = 10;        // change x to 10

    int &&r_ref = 200;   // r-value ref
    r_ref = 300;        // change r_ref to 300

    int &&x_ref = x;    // Compiler error
```

l-value reference parameters

```
int x {100};           // x is an l-value
```

```
void func(int &num); // A
```

```
func(x);           // calls A - x is an l-value
```

```
func(200);         // Error - 200 is an r-value
```

error: cannot bind non-const lvalue reference of type 'int&' to an rvalue of type 'int'

r-value reference parameters

```
int x {100};           // x is an l-value
```

```
void func(int &&num); // B
```

```
func(200);           // calls B - 200 is an r-value
```

```
func(x);             // ERROR - x is an l-value
```

error: cannot bind rvalue reference of type 'int&&' to lvalue of type 'int'

l-value and r-value reference parameters

```
int x {100};           // x is an l-value
```

```
void func(int &num);    // A
```

```
void func(int &&num);   // B
```

```
func(x);               // calls A - x is an l-value
```

```
func(200);             // calls B - 200 is an r-value
```

Example - Move class

```
class Move {  
private:  
    int *data;           // raw pointer  
public:  
    void set_data_value(int d) { *data = d; }  
    int get_data_value()     { return *data; }  
    Move(int d);             // Constructor  
    Move(const Move &source); // Copy Constructor  
    ~Move();                 // Destructor  
};
```


Move class copy constructor

```
Move::Move(const Move &source) {  
    data = new int;  
    *data = *source.data;  
}
```

Allocate storage and copy

Inefficient copying

```
Vector<Move> vec;
```

```
vec.push_back(Move{10});
```

```
vec.push_back(Move{20});
```

Copy Constructors will be called to copy the temps

Inefficient copying

```
Constructor for: 10
Constructor for: 10
Copy constructor - deep copy for: 10
Destructor freeing data for: 10
Constructor for: 20
Constructor for: 20
Copy constructor - deep copy for: 20
Constructor for: 10
Copy constructor - deep copy for: 10
Destructor freeing data for: 10
Destructor freeing data for: 20
```


What does it do?

- Instead of making a deep copy of the move constructor
 - 'moves' the resource
 - Simply copies the address of the resource from source to the current object
 - And, nulls out the pointer in the source pointer
- Very efficient

syntax - r-value reference

```
Type::Type (Type &&source) ;
```

```
Player::Player (Player &&source) ;
```

```
Move::Move (Move &&source) ;
```

Move class with move constructor

```
class Move {  
private:  
    int *data;           // raw pointer  
public:  
    void set_data_value(int d) { *data = d; }  
    int get_data_value()     { return *data; }  
    Move(int d);             // Constructor  
    Move(const Move &source); // Copy Constructor  
    Move(Move &&source);      // Move Constructor  
    ~Move();                 // Destructor  
};
```

Move class move constructor

```
Move::Move(Move &&source)
    : data{source.data} {
    source.data = nullptr;
}
```

'Steal' the data and then null out the source pointer

efficient

```
Vector<Move> vec;
```

```
vec.push_back(Move{10});
```

```
vec.push_back(Move{20});
```

Move Constructors will be called for the temp r-values

efficient

Constructor for: 10

Move constructor - moving resource: 10

Destructor freeing data for nullptr

Constructor for: 20

Move constructor - moving resource: 20

Move constructor - moving resource: 10

Destructor freeing data for nullptr

Destructor freeing data for nullptr

Destructor freeing data for: 10

Destructor freeing data for: 20

this pointer

- `this` is a reserved keyword
- Contains the address of the object - so it's a pointer to the object
- Can only be used in class scope
- All member access is done via the `this` pointer
- Can be used by the programmer
 - To access data member and methods
 - To determine if two objects are the same (more in the next section)
 - Can be dereferenced (`*this`) to yield the current object

this pointer

```
void Account::set_balance(double bal) {  
    balance = bal; // this->balance is implied  
}
```


this pointer

To disambiguate identifier use

```
void Account::set_balance(double balance) {  
    balance = balance;    // which balance? The parameter  
}
```

```
void Account::set_balance(double balance) {  
    this->balance = balance;    // Unambiguous  
}
```

this pointer

To determine object identity

- Sometimes its useful to know if two objects are the same object

```
int Account::compare_balance(const Account &other) {  
    if (this == &other)  
        std::cout << "The same objects" << std::endl;  
    ...  
}  
frank_account.compare_balance(frank_account);
```

- We'll use the `this` pointer again when we overload the assignment operator

Using const with classes

- Pass arguments to class member methods as `const`
- We can also create `const` objects
- What happens if we call member functions on `const` objects?
- `const`-correctness

Creating a const object

- `villain` is a const object so its attributes cannot change

```
const Player villain {"Villain", 100, 55};
```

Using const with classes

What happens when we call member methods on a const object?

```
const Player villain {"Villain", 100, 55};  
  
villain.set_name("Nice guy");           // ERROR  
  
std::cout << villain.get_name() << std::endl; // ERROR
```


Using const with classes

What happens when we call member methods on a const object?

```
const Player villain {"Villain", 100, 55};
```

```
void display_player_name(const Player &p) {  
    cout << p.get_name() << endl;  
}
```

```
display_player_name(villain);           // ERROR
```

Using const with classes

const methods

```
class Player {  
    private:  
        . . .  
    public:  
        std::string get_name() const;  
        . . .  
};
```

Using const with classes

const-correctness

```
const Player villain {"Villain", 100, 55};
```

```
villain.set_name("Nice guy");           // ERROR
```

```
std::cout << villain.get_name() << std::endl; // OK
```


Using const with classes

const methods

```
class Player {  
    private:  
        . . .  
    public:  
        std::string get_name() const;  
        // ERROR if code in get_name modifies this object  
        . . .  
};
```

Static Class Members

- Class data members can be declared as static
 - A single data member that belongs to the class, not the objects
 - Useful to store class-wide information
- Class functions can be declared as static
 - Independent of any objects
 - Can be called using the class name

Player class -static members

```
class Player {  
private:  
    static int num_players;  
  
public:  
    static int get_num_players();  
    . . .  
};
```

Player class – initialize the static data

Typically in Player.cpp

```
#include "Player.h"
```

```
int Player::num_players = 0;
```

Player class – implement static method

```
int Player::get_num_players() {  
    return num_players;  
}
```


Player class -update the constructor

```
Player::Player(std::string name_val, int health_val,  
int xp_val)  
    : name{name_val}, health{health_val}, xp{xp_val} {  
        ++num_players;  
    }
```

Player class - Destructor

```
Player::~~Player() {  
    --num_players;  
}
```

main.cpp

```
void display_active_players() {  
    cout << "Active players: "  
        << Player::get_num_players() << endl;  
}
```

```
int main() {  
    display_active_players();  
  
    Player obj1 {"Frank"};  
    display_active_players();  
    . . .  
}
```


Structs vs Classes

- In addition to define a `class` we can declare a `struct`
- `struct` comes from the C programming language
- Essentially the same as a `class` expect
 - members are `public` by default

class

```
class Person {  
    std::string name;  
    std::string get_name();  
};
```

```
Person p;  
p.name = "Frank";    // compiler error - private  
std::cout << p.get_name(); // compiler error - private
```

struct

```
struct Person {  
    std::string name;  
    std::string get_name(); // Why if name is public?  
};
```

```
Person p;  
p.name = "Frank";    // OK - public  
std::cout << p.get_name(); // OK - public
```

Some general guidelines

- `struct`

- Use `struct` for passive objects with public access
- Don't declare methods in `struct`

- `class`

- Use `class` for active objects with private access
- Implement getters/setters as needed
- Implement member methods as needed

Friends of a Class

- Friend

- A function or class that has access to private class member
- And, that function or class is NOT a member of the class it is accessing

- Function

- Can be regular non-member functions
- Can be member methods of another class

- Class

- Another class can have access to private class members

Friends of a Class

- Friendship must be granted NOT taken
 - Declared explicitly in the class that is granting friendship
 - Declared in the function prototype with the keyword `friend`
- Friendship is not symmetric
 - Must be explicitly granted
 - if A is a friend of B
 - B is NOT a friend of A
- Friendship is not transitive
 - Must be explicitly granted
 - if A is a friend of B AND
 - B is a friend of C
 - then A is NOT a friend of C

non-member function

```
class Player {  
    friend void display_player(Player &p) ;  
    std::string name;  
    int health;  
    int xp;  
public:  
    . . .  
};
```

non-member function

```
void display_player(Player &p) {  
    std::cout << p.name << std::endl;  
    std::cout << p.health << std::endl;  
    std::cout << p.xp << std::endl;  
}
```

`display_player` may also change private data members

member function of another class

```
class Player {  
    friend void Other_class::display_player(Player &p) ;  
    std::string name;  
    int health;  
    int xp;  
public:  
    . . .  
};
```

member function of another class

```
class Other_class {  
    . . .  
public:  
    void display_player(Player &p) {  
        std::cout << p.name << std::endl;  
        std::cout << p.health << std::endl;  
        std::cout << p.xp << std::endl;  
    }  
};
```

Another class as a friend

```
class Player {  
    friend class Other_class;  
    std::string name;  
    int health;  
    int xp;  
public:  
    . . .  
};
```

Section Overview

Operator Overloading

- What is Operator Overloading?
- Overloading the assignment operator (=)
 - Copy semantics
 - Move semantics
- Overloading operators as member functions
- Overloading operators as global functions
- Overloading stream insertion (<<) and extraction operators (>>)

Operator Overloading

What is Operator Overloading?

- Using traditional operators such as +, =, *, etc. with user-defined types
- Allows user defined types to behave similar to built-in types
- Can make code more readable and writable
- Not done automatically (except for the assignment operator)
They must be explicitly defined

Operator Overloading

What is Operator Overloading?

Suppose we have a Number class that models any number

- Using functions:

```
Number result = multiply(add(a,b) ,divide(c,d) ) ;
```

- Using member methods:

```
Number result = (a.add(b) ) .multiply(c.divide(d) ) ;
```

Operator Overloading

What is Operator Overloading?

Suppose we have a Number class that models any number

- Using overloaded operators

```
Number result = (a+b) * (c/d) ;
```

Operator Overloading

What operators can be overloaded?

- The majority of C++'s operators can be overloaded
- The following operators cannot be overload

operator
::
::?
.*
.
sizeof

Operator Overloading

Some basic rules

- Precedence and Associativity cannot be changed
- 'arity' cannot be changed (i.e. can't make the division operator unary)
- Can't overload operators for primitive type (e.g. `int`, `double`, etc.)
- Can't create new operators
- `[]`, `()`, `->`, and the assignment operator `=` **must** be declared as member methods
- Other operators can be declared as member methods or global functions

Operator Overloading

Some examples

- **int**

```
a = b + c  
a < b  
std::cout << a
```

- **double**

```
a = b + c  
a < b  
std::cout << a
```

- **long**

```
a = b + c  
a < b  
std::cout << a
```

- **std::string**

```
s1 = s2 + s3  
s1 < s2  
std::cout << s1
```

- **Mystring**

```
s1 = s2 + s3  
s1 < s2  
s1 == s2  
std::cout << s1
```

- **Player**

```
p1 < p2  
p1 == p2  
std::cout << p1
```

Operator Overloading

Mystring class declaration

```
class Mystring {  
  
private:  
    char *str; // C-style string  
  
public:  
    Mystring();  
    Mystring(const char *s);  
    Mystring(const Mystring &source);  
    ~Mystring();  
    void display() const;  
    int get_length() const;  
    const char *get_str() const;  
};
```

Operator Overloading

Copy assignment operator (=)

- C++ provides a default assignment operator used for assigning one object to another

```
Mystring s1 {"Frank"};  
Mystring s2 = s1;    // NOT assignment  
                  // same as Mystring s2{s1};
```

```
s2 = s1;           // assignment
```

- Default is memberwise assignment (shallow copy)
 - If we have raw pointer data member we must deep copy

Operator Overloading

Overloading the copy assignment operator (deep copy)

```
Type &Type::operator=(const Type &rhs) ;
```

```
Mystring &Mystring::operator=(const Mystring &rhs) ;
```

```
s2 = s1;           // We write this
```

```
s2.operator=(s1) ;    // operator= method is called
```


Operator Overloading

Overloading the copy assignment operator (deep copy)

```
Mystring &Mystring::operator=(const Mystring &rhs) {  
    if (this == &rhs)  
        return *this;  
  
    delete [] str;  
    str = new char[std::strlen(rhs.str) + 1];  
    std::strcpy(str, rhs.str);  
  
    return *this;  
}
```

Operator Overloading

Overloading the copy assignment operator – steps for deep copy

- Check for self assignment

```
if (this == &rhs)    // p1 = p1;  
    return *this;    // return current object
```

- Deallocate storage for `this->str` since we are overwriting it

```
delete [] str;
```

Operator Overloading

Overloading the copy assignment operator – steps for deep copy

- Allocate storage for the deep copy

```
str = new char[std::strlen(rhs.str) + 1];
```


Operator Overloading

Overloading the copy assignment operator – steps for deep copy

- Perform the copy

```
std::strcpy(str, rhs.str);
```

- Return the current by reference to allow chain assignment

```
return *this;
```

```
// s1 = s2 = s3;
```

Operator Overloading

Move assignment operator (=)

- You can choose to overload the move assignment operator
 - C++ will use the copy assignment operator if necessary

```
Mystring s1;
```

```
s1 = Mystring {"Frank"}; // move assignment
```

- If we have raw pointer we should overload the move assignment operator for efficiency

Operator Overloading

Overloading the Move assignment operator

```
Type &Type::operator=(Type &&rhs) ;
```

```
Mystring &Mystring::operator=(Mystring &&rhs) ;
```

```
s1 = Mystring{"Joe"}; // move operator= called
```

```
s1 = "Frank";          // move operator= called
```

Operator Overloading

Overloading the Move assignment operator

```
Mystring &Mystring::operator=(Mystring &&rhs) {  
  
    if (this == &rhs)        // self assignment  
        return *this;        // return current object  
  
    delete [] str;           // deallocate current storage  
    str = rhs.str;           // steal the pointer  
  
    rhs.str = nullptr;       // null out the rhs object  
  
    return *this;            // return current object  
}
```

Operator Overloading

Overloading the Move assignment operator – steps

- Check for self assignment

```
if (this == &rhs)
    return *this;        // return current object
```

- Deallocate storage for `this->str` since we are overwriting it

```
delete [] str;
```


Operator Overloading

Overloading the Move assignment operator – steps for deep copy

- Steal the pointer from the rhs object and assign it to this->str

```
str = rhs.str;
```

- Null out the rhs pointer

```
rhs.str = nullptr;
```

- Return the current object by reference to allow chain assignment

```
return *this;
```

Operator Overloading

Unary operators as member methods (++ , -- , - , !)

```
ReturnType Type::operatorOp() ;
```

```
Number Number::operator-() const;
```

```
Number Number::operator++();           // pre-increment
```

```
Number Number::operator++(int);        // post-increment
```

```
bool Number::operator!() const;
```

```
Number n1 {100};
```

```
Number n2 = -n1;           // n1.operator-()
```

```
n2 = ++n1;                 // n1.operator++()
```

```
n2 = n1++;                 // n1.operator++(int)
```

Operator Overloading

Mystring operator- make lowercase

```
Mystring larry1 {"LARRY"};
Mystring larry2;

larry1.display();           // LARRY

larry2 = -larry1;           // larry1.operator-()

larry1.display();           // LARRY
larry2.display();           // larry
```


Operator Overloading

Mystring operator- make lowercase

```
Mystring Mystring::operator- () const {  
    char *buff = new char[std::strlen(str) + 1];  
    std::strcpy(buff, str);  
    for (size_t i=0; i<std::strlen(buff); i++)  
        buff[i] = std::tolower(buff[i]);  
    Mystring temp {buff};  
    delete [] buff;  
    return temp;  
}
```

Operator Overloading

Binary operators as member methods (+, -, ==, !=, <, >, etc.)

```
ReturnType Type::operatorOp(const Type &rhs);
```

```
Number Number::operator+(const Number &rhs) const;
```

```
Number Number::operator-(const Number &rhs) const;
```

```
bool Number::operator==(const Number &rhs) const;
```

```
bool Number::operator<(const Number &rhs) const;
```

```
Number n1 {100}, n2 {200};
```

```
Number n3 = n1 + n2; // n1.operator+(n2)
```

```
n3 = n1 - n2; // n1.operator-(n2)
```

```
if (n1 == n2) . . . // n1.operator==(n2)
```

Operator Overloading

Mystring operator==

```
bool Mystring::operator==(const Mystring &rhs) const {  
    if (std::strcmp(str, rhs.str) == 0)  
        return true;  
    else  
        return false;  
}  
  
// if (s1 == s2)           // s1 and s2 are Mystring objects
```

Operator Overloading

Mystring operator+ (concatenation)

```
Mystring larry {"Larry"};
Mystring moe {"Moe"};
Mystring stooges {" is one of the three stooges"};

Mystring result = larry + stooges;
                  // larry.operator+(stooges);

result = moe + " is also a stooge";
          // moe.operator+("is also a stooge");

result = "Moe" + stooges; // "Moe".operator+(stooges) ERROR
```

Operator Overloading

Mystring operator+ (concatenation)

```
Mystring Mystring::operator+(const Mystring &rhs) const {  
    size_t buff_size = std::strlen(str) +  
                        std::strlen(rhs.str) + 1;  
    char *buff = new char[buff_size];  
    std::strcpy(buff, str);  
    std::strcat(buff, rhs.str);  
    Mystring temp {buff};  
    delete [] buff;  
    return temp;  
}
```


Operator Overloading

Unary operators as global functions (++ , -- , - , !)

```
ReturnType operatorOp(Type &obj);
```

```
Number operator-(const Number &obj);
```

```
Number operator++(Number &obj);           // pre-increment
```

```
Number operator++(Number &obj, int);      // post-increment
```

```
bool operator!(const Number &obj);
```

```
Number n1 {100};
```

```
Number n2 = -n1;           // operator-(n1)
```

```
n2 = ++n1;                 // operator++(n1)
```

```
n2 = n1++;                 // operator++(n1,int)
```

Operator Overloading

Mystring operator- make lowercase

```
Mystring larry1 {"LARRY"};
Mystring larry2;

larry1.display();           // LARRY

larry2 = -larry1;           // operator-(larry1)

larry1.display();           // LARRY
larry2.display();           // larry
```

Operator Overloading

Mystring operator-

- Often declared as **friend** functions in the class declaration

```
Mystring operator-(const Mystring &obj) {  
    char *buff = new char[std::strlen(obj.str) + 1];  
    std::strcpy(buff, obj.str);  
    for (size_t i=0; i<std::strlen(buff); i++)  
        buff[i] = std::tolower(buff[i]);  
    Mystring temp {buff};  
    delete [] buff;  
    return temp;  
}
```


Operator Overloading

Binary operators as global functions (+, -, ==, !=, <, >, etc.)

```
ReturnType operatorOp(const Type &lhs, const Type &rhs);
```

```
Number operator+(const Number &lhs, const Number &rhs);
```

```
Number operator-(const Number &lhs, const Number &rhs);
```

```
bool operator==(const Number &lhs, const Number &rhs);
```

```
bool operator<(const Number &lhs, const Number &rhs);
```

```
Number n1 {100}, n2 {200};
```

```
Number n3 = n1 + n2; // operator+(n1,n2)
```

```
n3 = n1 - n2; // operator-(n1,n2)
```

```
if (n1 == n2) . . . // operator==(n1,n2)
```

Operator Overloading

Mystring operator==

```
bool operator==(const Mystring &lhs, const Mystring &rhs) {  
    if (std::strcmp(lhs.str, rhs.str) == 0)  
        return true;  
    else  
        return false;  
}
```

- If declared as a friend of `Mystring` can access private `str` attribute
- Otherwise we must use getter methods

Operator Overloading

Mystring operator+ (concatenation)

```
Mystring larry {"Larry"};
Mystring moe {"Moe"};
Mystring stooges {" is one of the three stooges"};

Mystring result = larry + stooges;
                  // operator+(larry, stooges);

result = moe + " is also a stooge";
          // operator+(moe, "is also a stooge");

result = "Moe" + stooges; // OK with non-member functions
```

Operator Overloading

Mystring operator+ (concatenation)

```
Mystring operator+(const Mystring &lhs, const Mystring &rhs) {  
    size_t buff_size = std::strlen(lhs.str) +  
                        std::strlen(rhs.str) + 1;  
    char *buff = new char[buff_size];  
    std::strcpy(buff, lhs.str);  
    std::strcat(buff, rhs.str);  
    Mystring temp {buff};  
    delete [] buff;  
    return temp;  
}
```

Operator Overloading

stream insertion and extraction operators (<<, >>)

```
Mystring larry {"Larry"};
```

```
cout << larry << endl;    // Larry
```

```
Player hero {"Hero", 100, 33};
```

```
cout << hero << endl;    // {name: Hero, health: 100, xp:33}
```


Operator Overloading

stream insertion and extraction operators (<<, >>)

```
Mystring larry;
```

```
cin >> larry;
```

```
Player hero;
```

```
cin >> hero;
```

Operator Overloading

stream insertion and extraction operators (<<, >>)

- Doesn't make sense to implement as member methods
 - Left operand must be a user-defined class
 - Not the way we normally use these operators

```
Mystring larry;  
larry << cout;           // huh?
```

```
Player hero;  
hero >> cin;             // huh?
```

Operator Overloading

stream insertion operator (<<)

```
std::ostream &operator<<(std::ostream &os, const Mystring &obj) {  
    os << obj.str;    // if friend function  
    // os << obj.get_str();    // if not friend function  
    return os;  
}
```

- Return a reference to the `ostream` so we can keep inserting
- Don't return `ostream` by value!

Operator Overloading

stream extraction operator (>>)

```
std::istream &operator>>(std::istream &is, Mystring &obj) {  
    char *buff = new char[1000];  
    is >> buff;  
    obj = Mystring(buff); // If you have copy or move assignment  
    delete [] buff;  
    return is;  
}
```

- Return a reference to the `istream` so we can keep inserting
- Update the object passed in

Section Overview

Inheritance

- What is Inheritance?
 - Why is it useful?
- Terminology and Notation
- Inheritance vs. Composition
- Deriving classes from existing classes
 - Types of inheritance
- Protected members and class access
- Constructors and Destructors
 - Passing arguments to base class constructors
 - Order of constructor and destructors calls
- Redefining base class methods
- Class Hierarchies
- Multiple Inheritance

Inheritance

What is it and why is it used?

- Provides a method for creating new classes from existing classes
- The new class contains the data and behaviors of the existing class
- Allow for reuse of existing classes
- Allows us to focus on the common attributes among a set of classes
- Allows new classes to modify behaviors of existing classes to make it unique
 - Without actually modifying the original class

Inheritance

Related classes

- Player, Enemy, Level Boss, Hero, Super Player, etc.
- Account, Savings Account, Checking Account, Trust Account, etc.
- Shape, Line, Oval, Circle, Square, etc.
- Person, Employee, Student, Faculty, Staff, Administrator, etc.

Inheritance

Accounts

- Account
 - **balance, deposit, withdraw, . . .**
- Savings Account
 - **balance, deposit, withdraw**, interest rate, . . .
- Checking Account
 - **balance, deposit, withdraw**, minimum balance, per check fee, . . .
- Trust Account
 - **balance, deposit, withdraw**, interest rate, . . .

Inheritance

Accounts – without inheritance – code duplication

```
class Account {  
    // balance, deposit, withdraw, . . .  
};  
  
class Savings_Account {  
    // balance, deposit, withdraw, interest rate, . . .  
};  
  
class Checking_Account {  
    // balance, deposit, withdraw, minimum balance, per check fee, . . .  
};  
  
class Trust_Account {  
    // balance, deposit, withdraw, interest rate, . . .  
};
```

Inheritance

Accounts – with inheritance – code reuse

```
class Account {  
    // balance, deposit, withdraw, . . .  
};  
  
class Savings_Account : public Account {  
    // interest rate, specialized withdraw, . . .  
};  
  
class Checking_Account : public Account {  
    // minimum balance, per check fee, specialized withdraw, . . .  
};  
  
class Trust_Account : public Account {  
    // interest rate, specialized withdraw, . . .  
};
```

Inheritance

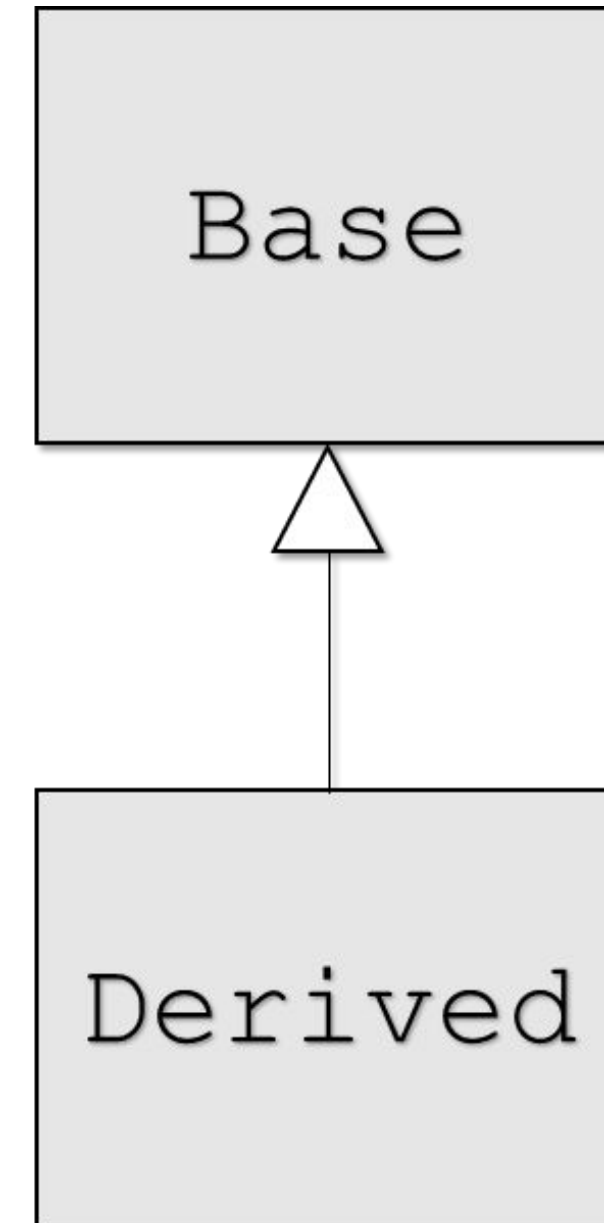
Terminology

- Inheritance
 - Process of creating new classes from existing classes
 - Reuse mechanism
- Single Inheritance
 - A new class is created from another 'single' class
- Multiple Inheritance
 - A new class is created from two (or more) other classes

Inheritance

Terminology

- Base class (parent class, super class)
 - The class being extended or inherited from
- Derived class (child class, sub class)
 - The class being created from the Base class
 - Will inherit attributes and operations from Base class



Inheritance

Terminology

- “Is-A” relationship
 - Public inheritance
 - Derived classes are sub-types of their Base classes
 - Can use a derived class object wherever we use a base class object
- Generalization
 - Combining similar classes into a single, more general class based on common attributes
- Specialization
 - Creating new classes from existing classes providing more specialized attributes or operations
- Inheritance or Class Hierarchies
 - Organization of our inheritance relationships

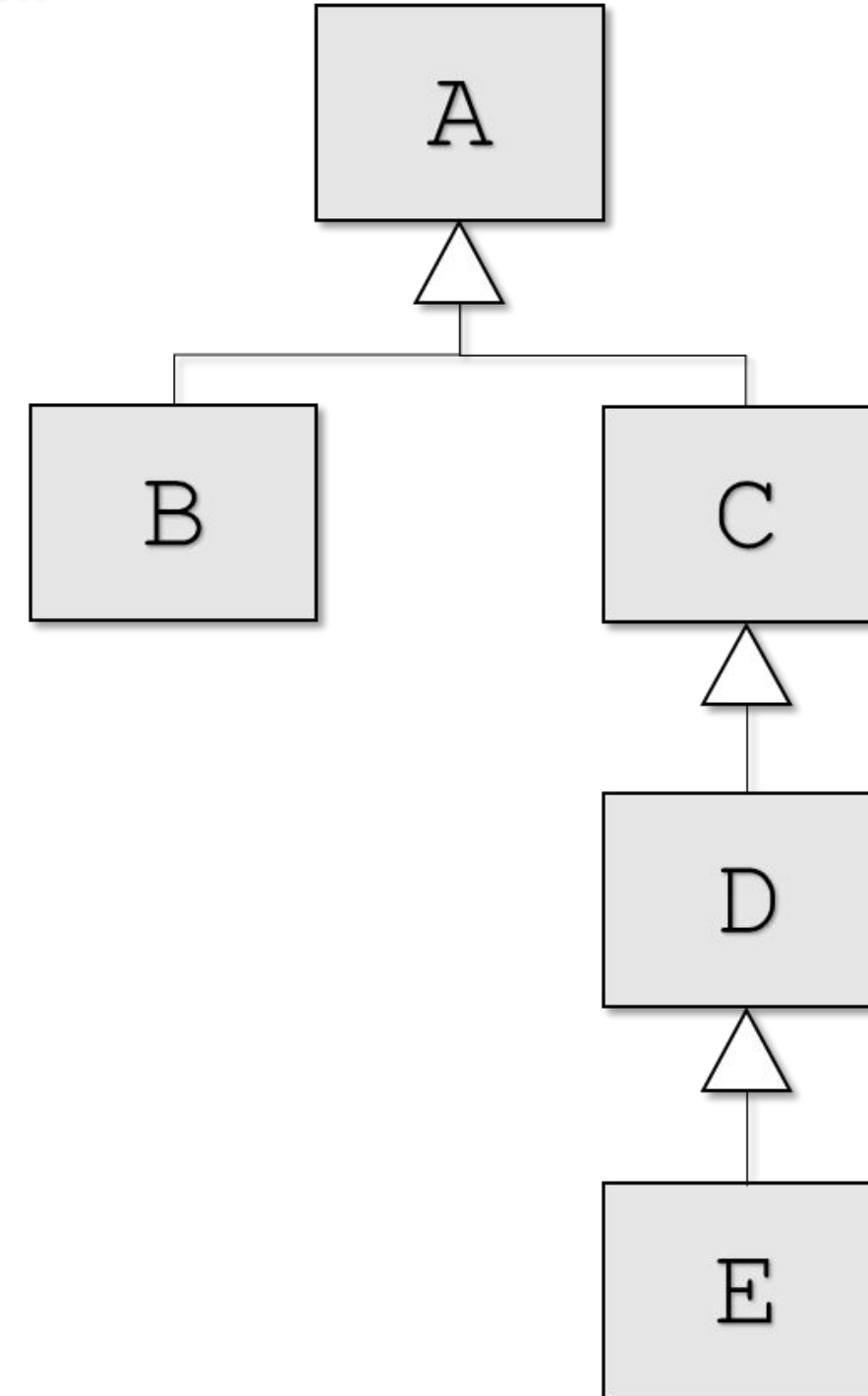
Inheritance

Class hierarchy

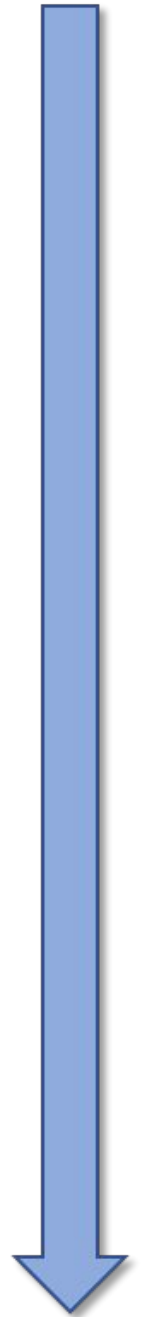
Classes:

- A
- B is derived from A
- C is derived from A
- D is derived from C
- E is derived from D

Generalization



Specialization

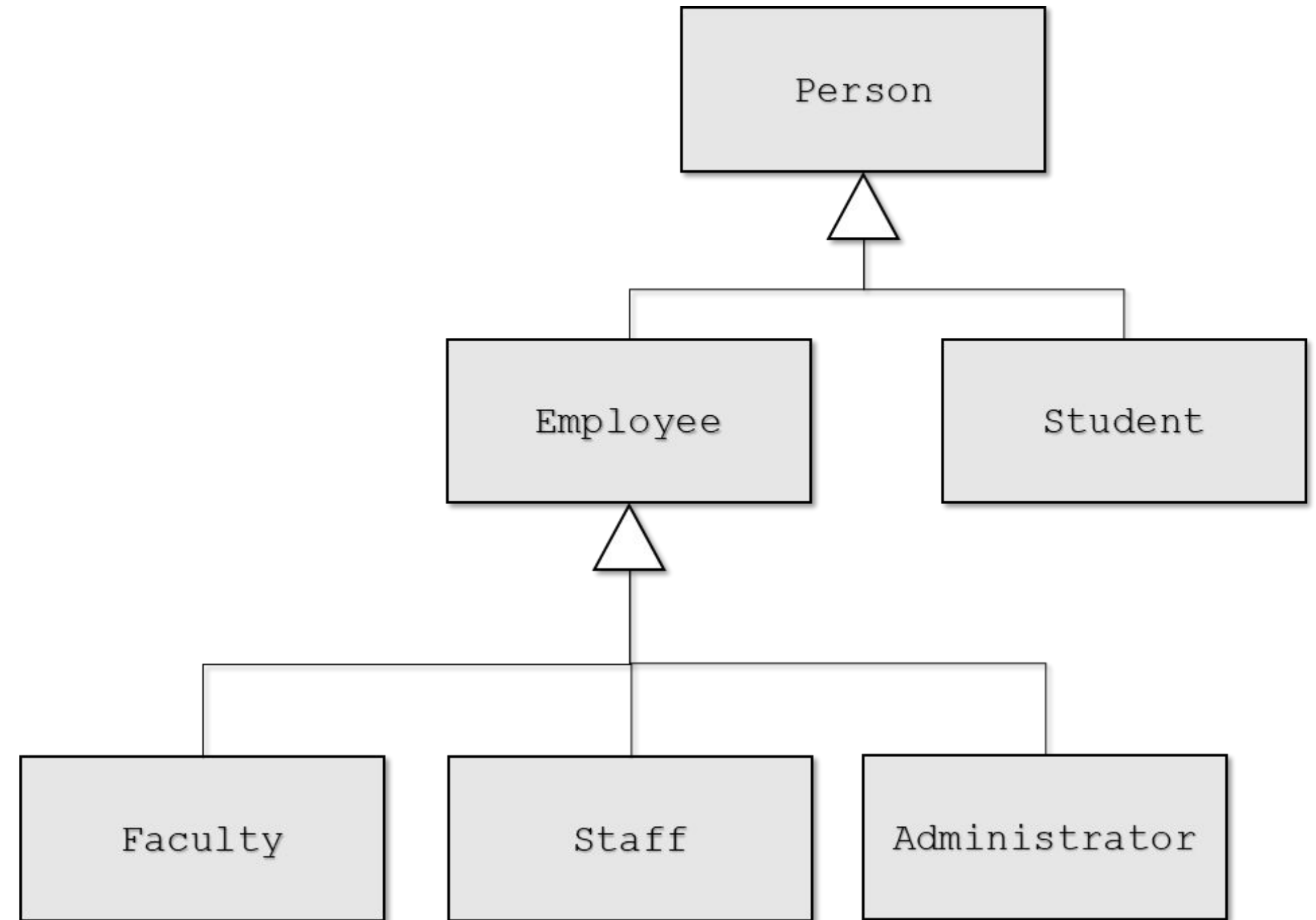


Inheritance

Class hierarchy

Classes:

- Person
- Employee is derived from Person
- Student is derived from Person
- Faculty is derived from Employee
- Staff is derived from Employee
- Administrator is derived from Employee



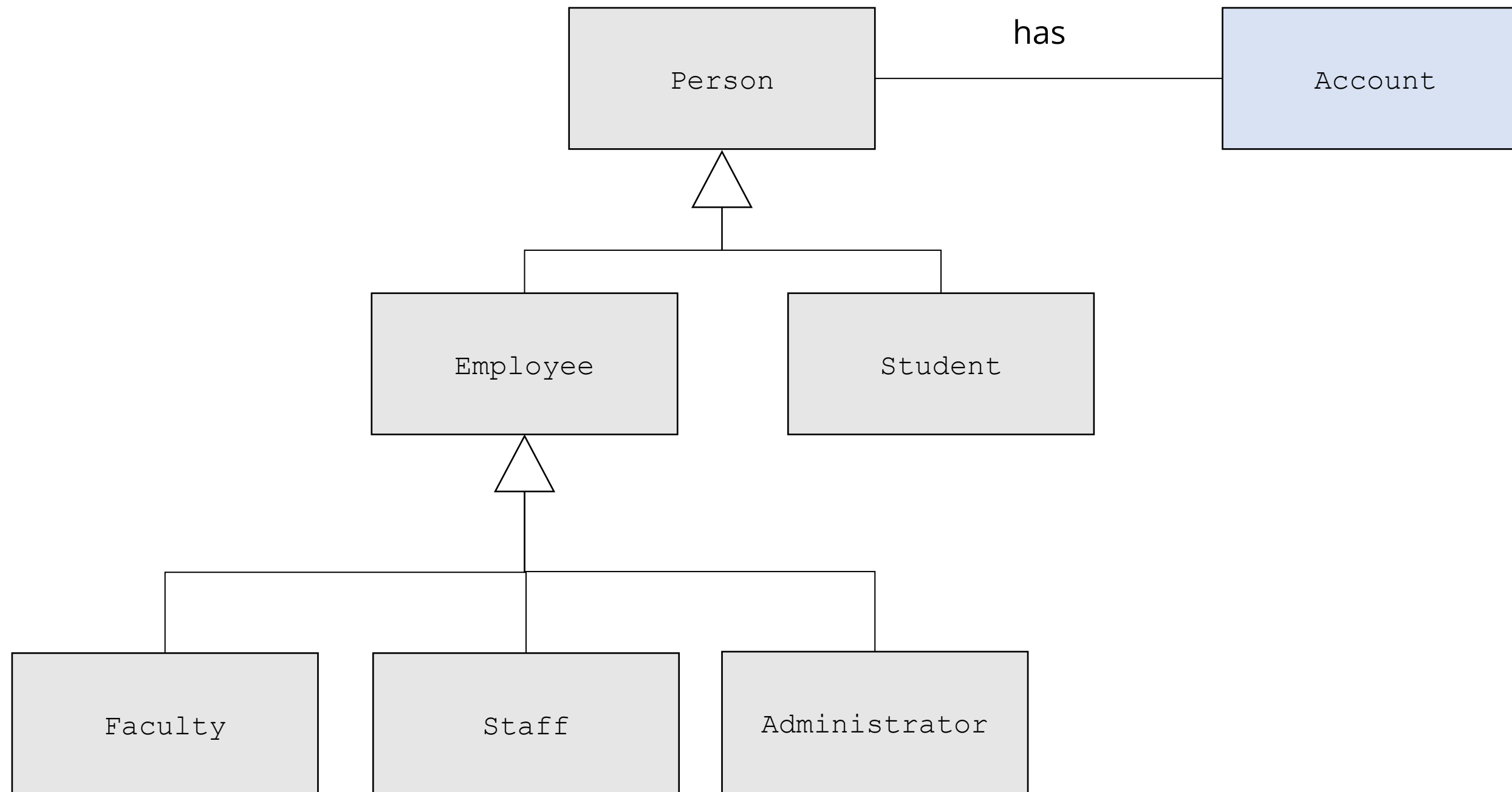
Inheritance

Public Inheritance vs. Composition

- Both allow reuse of existing classes
- Public Inheritance
 - “is-a” relationship
 - Employee ‘is-a’ Person
 - Checking Account ‘is-a’ Account
 - Circle “is-a” Shape
- Composition
 - “has-a” relationship
 - Person “has a” Account
 - Player “has-a” Special Attack
 - Circle “has-a” Location

Inheritance

Public Inheritance vs. Composition



Inheritance

Public Inheritance vs. Composition

```
class Person {  
private:  
    std::string name; // has-a name  
    Account account; // has-a account  
};
```

Deriving classes from exiting classes

C++ derivation syntax

```
class Base {  
    // Base class members . . .  
};
```

```
class Derived: access-specifier Base {  
    // Derived class members . . .  
};
```

Access-specifier can be: `public`, `private`, or `protected`

Deriving classes from exiting classes

Types of inheritance in C++

- `public`
 - Most common
 - Establishes '**is-a**' relationship between Derived and Base classes
- `private` and `protected`
 - Establishes "derived class **has a** base class" relationship
 - "Is implemented in terms of" relationship
 - Different from composition

Deriving classes from exiting classes

C++ derivation syntax

```
class Account {  
    // Account class members . . .  
};  
  
class Savings_Account: public Account {  
    // Savings_Account class members . . .  
};
```

Savings_Account 'is-a' Account

Deriving classes from exiting classes

C++ creating objects

```
Account account {};
```

```
Account *p_account = new Account();
```

```
account.deposit(1000.0);
```

```
p_account->withdraw(200.0);
```

```
delete p_account;
```

Deriving classes from exiting classes

C++ creating objects

```
Savings_Account sav_account {};
```

```
Savings_Account *p_sav_account = new Savings_Account();
```

```
sav_account.deposit(1000.0);
```

```
p_sav_account->withdraw(200.0);
```

```
delete p_sav_account;
```

Protected Members and Class Access

The protected class member modifier

```
class Base {
```

```
    protected:
```

```
    // protected Base class members . . .
```

```
};
```

- Accessible from the Base class itself
- Accessible from classes Derived from Base
- Not accessible by objects of Base or Derived

Protected Members and Class Access

The protected class member modifier

```
class Base {  
    public:  
        int a;    // public Base class members . . .  
  
    protected:  
        int b; // protected Base class members . . .  
  
    private:  
        int c; // private Base class members . . .  
  
};
```

Deriving classes from existing classes

Access with **public** inheritance

Base Class

```
public: a  
protected: b  
private: c
```

**public
inheritance**

Access in Derived Class

```
public: a  
protected: b  
c : no access
```

Deriving classes from existing classes

Access with **protected** inheritance

Base Class

```
public: a  
protected: b  
private: c
```

**protected
inheritance**

Access in Derived Class

```
protected: a  
protected: b  
c : no access
```


Deriving classes from existing classes

Access with **private** inheritance

Base Class

```
public: a  
protected: b  
private: c
```

**private
inheritance**

Access in Derived Class

```
private: a  
private: b  
c : no access
```

Constructors and Destructors

Constructors and class initialization

- A Derived class inherits from its Base class
- The Base part of the Derived class MUST be initialized BEFORE the Derived class is initialized
- When a Derived object is created
 - Base class constructor executes then
 - Derived class constructor executes

Constructors and Destructors

Constructors and class initialization

```
class Base {  
public:  
    Base() { cout << "Base constructor" << endl; }  
};  
  
class Derived : public Base {  
public:  
    Derived() { cout << "Derived constructor " << endl; }  
};
```

Constructors and Destructors

Constructors and class initialization

Output

Base base;

Base constructor

Derived derived;

Base constructor

Derived constructor

Constructors and Destructors

Destructors

- Class destructors are invoked in the reverse order as constructors
- The Derived part of the Derived class MUST be destroyed BEFORE the Base class destructor is invoked
- When a Derived object is destroyed
 - Derived class destructor executes then
 - Base class destructor executes
 - Each destructor should free resources allocated in it's own constructors

Constructors and Destructors

Destructors

```
class Base {  
public:  
    Base() { cout << "Base constructor" << endl; }  
    ~Base() { cout << "Base destructor" << endl; }  
};  
  
class Derived : public Base {  
public:  
    Derived() { cout << "Derived constructor " << endl; }  
    ~Derived() { cout << "Derived destructor " << endl; }  
};
```

Constructors and Destructors

Destructors and class initialization

```
Base base;
```

Output

```
Base constructor  
Base destructor
```

```
Derived derived;
```

```
Base constructor  
Derived constructor  
Derived destructor  
Base destructor
```

Constructors and Destructors

Constructors and class initialization

- A Derived class does NOT inherit
 - Base class constructors
 - Base class destructor
 - Base class overloaded assignment operators
 - Base class friend functions
- However, the derived class constructors, destructors, and overloaded assignment operators can invoke the base-class versions
- C++11 allows explicit inheritance of base 'non-special' constructors with
 - `using Base::Base;` anywhere in the derived class declaration
 - Lots of rules involved, often better to define constructors yourself

Inheritance

Passing arguments to base class constructors

- The Base part of a Derived class must be initialized first
- How can we control exactly which Base class constructor is used during initialization?
- We can invoke the whichever Base class constructor we wish in the initialization list of the Derived class

Inheritance

Passing arguments to base class constructors

```
class Base {  
public:  
    Base();  
    Base(int);  
    . . .  
};
```

```
Derived::Derived(int x)  
    : Base(x), {optional initializers for Derived} {  
    // code  
}
```

Constructors and Destructors

Constructors and class initialization

```
class Base {  
    int value;  
public:  
    Base() : value{0} {  
        cout << "Base no-args constructor" << endl;  
    }  
    Base(int x) : value{x} {  
        cout << "int Base constructor" << endl;  
    }  
};
```

Constructors and Destructors

Constructors and class initialization

```
class Derived : public Base {  
    int doubled_value;  
public:  
    Derived() : Base{}, doubled_value{0} {  
        cout << "Derived no-args constructor " << endl;  
    }  
    Derived(int x) : Base{x}, doubled_value {x*2} {  
        cout << "int Derived constructor " << endl;  
    }  
};
```

Constructors and Destructors

Constructors and class initialization

Output

```
Base base;
```

```
Base no-args constructor
```

```
Base base{100};
```

```
int Base constructor
```

```
Derived derived;
```

```
Base no-args constructor
```

```
Derived no-args constructor
```

```
Derived derived{100};
```

```
int Base constructor
```

```
int Derived constructor
```


Inheritance

Copy/Move constructors and overloaded operator=

- Not inherited from the Base class
- You may not need to provide your own
 - Compiler-provided versions may be just fine
- We can explicitly invoke the Base class versions from the Derived class

Inheritance

Copy constructor

- Can invoke Base copy constructor explicitly
 - Derived object '*other*' will be **sliced**

```
Derived::Derived(const Derived &other)  
    : Base(other), {Derived initialization list}  
{  
    // code  
}
```

Constructors and Destructors

Copy Constructors

```
class Base {  
    int value;  
public:  
    // Same constructors as previous example  
  
    Base(const Base &other) :value{other.value} {  
    cout << "Base copy constructor" << endl;  
    }  
};
```


Constructors and Destructors

Copy Constructors

```
class Derived : public Base {  
    int doubled_value;  
public:  
    // Same constructors as previous example  
  
    Derived(const Derived &other)  
: Base(other), doubled_value {other.doubled_value} {  
    cout << "Derived copy constructor " << endl;  
}  
};
```

Constructors and Destructors

operator=

```
class Base {  
    int value;  
public:  
    // Same constructors as previous example  
    Base &operator=(const Base &rhs) {  
        if (this != &rhs) {  
            value = rhs.value; // assign  
        }  
        return *this;  
    }  
};
```

Constructors and Destructors

operator=

```
class Derived : public Base {  
    int doubled_value;  
public:  
    // Same constructors as previous example  
    Derived &operator=(const Derived &rhs) {  
        if (this != &rhs) {  
            Base::operator=(rhs);        // Assign Base part  
            doubled_value = rhs.doubled_value; // Assign Derived part  
        }  
        return *this;  
    }  
};
```

Inheritance

Copy/Move constructors and overloaded operator=

- Often you do not need to provide your own
- If you **DO NOT** not define them in Derived
 - then the compiler will create them and automatically and call the base class's version
- If you **DO** provide Derived versions
 - then **YOU** must invoke the Base versions **explicitly** yourself
- Be careful with raw pointers
 - Especially if Base and Derived each have raw pointers
 - Provide them with deep copy semantics

Inheritance

Using and redefining Base class methods

- Derived class can directly invoke Base class methods
- Derived class can **override** or **redefine** Base class methods
- Very powerful in the context of polymorphism (next section)

Inheritance

Using and redefining Base class methods

```
class Account {  
public:  
    void deposit(double amount) { balance += amount; }  
};  
  
class Savings_Account: public Account {  
public:  
    void deposit(double amount) { // Redefine Base class method  
        amount += some_interest;  
        Account::deposit(amount); // invoke call Base class method  
    }  
};
```

Inheritance

Static binding of method calls

- Binding of which method to use is done at compile time
 - Default binding for C++ is static
 - Derived class objects will use `Derived::deposit`
 - But, we can explicitly invoke `Base::deposit` from `Derived::deposit`
 - OK, but limited – much more powerful approach is dynamic binding which we will see in the next section

Inheritance

Static binding of method calls

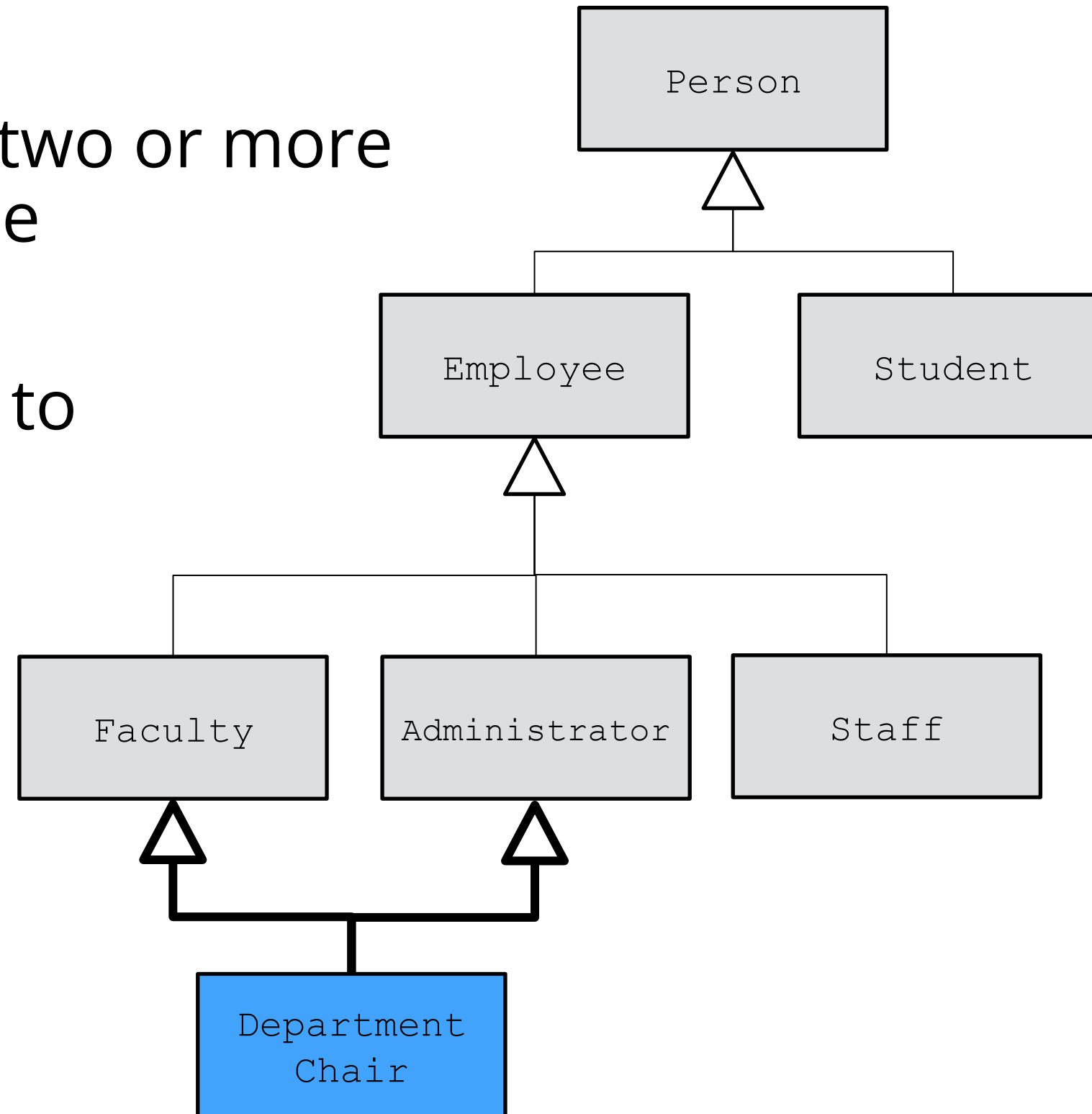
```
Base b;  
b.deposit(1000.0);           // Base::deposit
```

```
Derived d;  
d.deposit(1000.0);          // Derived::deposit
```

```
Base *ptr = new Derived();  
ptr->deposit(1000.0);        // Base::deposit ????
```


Multiple Inheritance

- A derived class inherits from two or more Base classes at the same time
- The Base classes may belong to unrelated class hierarchies
- A Department Chair
 - Is-A Faculty and
 - Is-A Administrator



Multiple Inheritance

C++ Syntax

```
class Department_Chair:  
    public Faculty, public Administrator {  
    . . .  
};
```

- Beyond the scope of this course
- Some compelling use-cases
- Easily misused
- Can be very complex

Section Overview

Polymorphism and Inheritance

- What is Polymorphism?
- Using base class pointers
- Static vs. dynamic binding
- Virtual functions
- Virtual destructors
- The override and final specifiers
- Using base class references
- Pure virtual functions and abstract classes
- Abstract classes as interfaces

Section Overview

Polymorphism and Inheritance

- What is Polymorphism?
- Using base class pointers
- Static vs. dynamic binding
- Virtual functions
- Virtual destructors
- The override and final specifiers
- Using base class references
- Pure virtual functions and abstract classes
- Abstract classes as interfaces

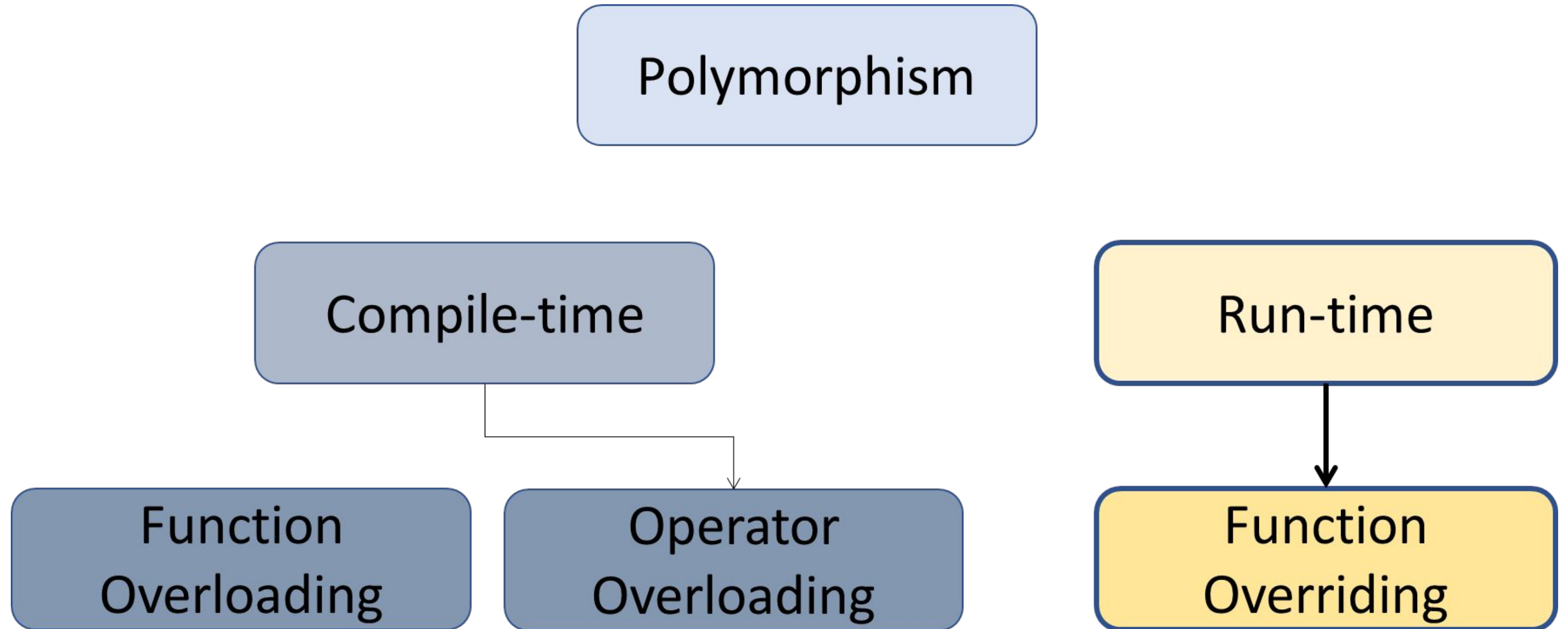
Polymorphism

What is Polymorphism?

- Fundamental to Object-Oriented Programming
- Polymorphism
 - Compile-time / early binding / static binding
 - **Run-time / late binding / dynamic binding**
- Runtime polymorphism
 - Being able to assign different meanings to the same function at run-time
- Allows us to program more abstractly
 - Think general vs. specific
 - Let C++ figure out which function to call at run-time
- Not the default in C++, run-time polymorphism is achieved via
 - Inheritance
 - Base class pointers or references
 - virtual functions

Polymorphism

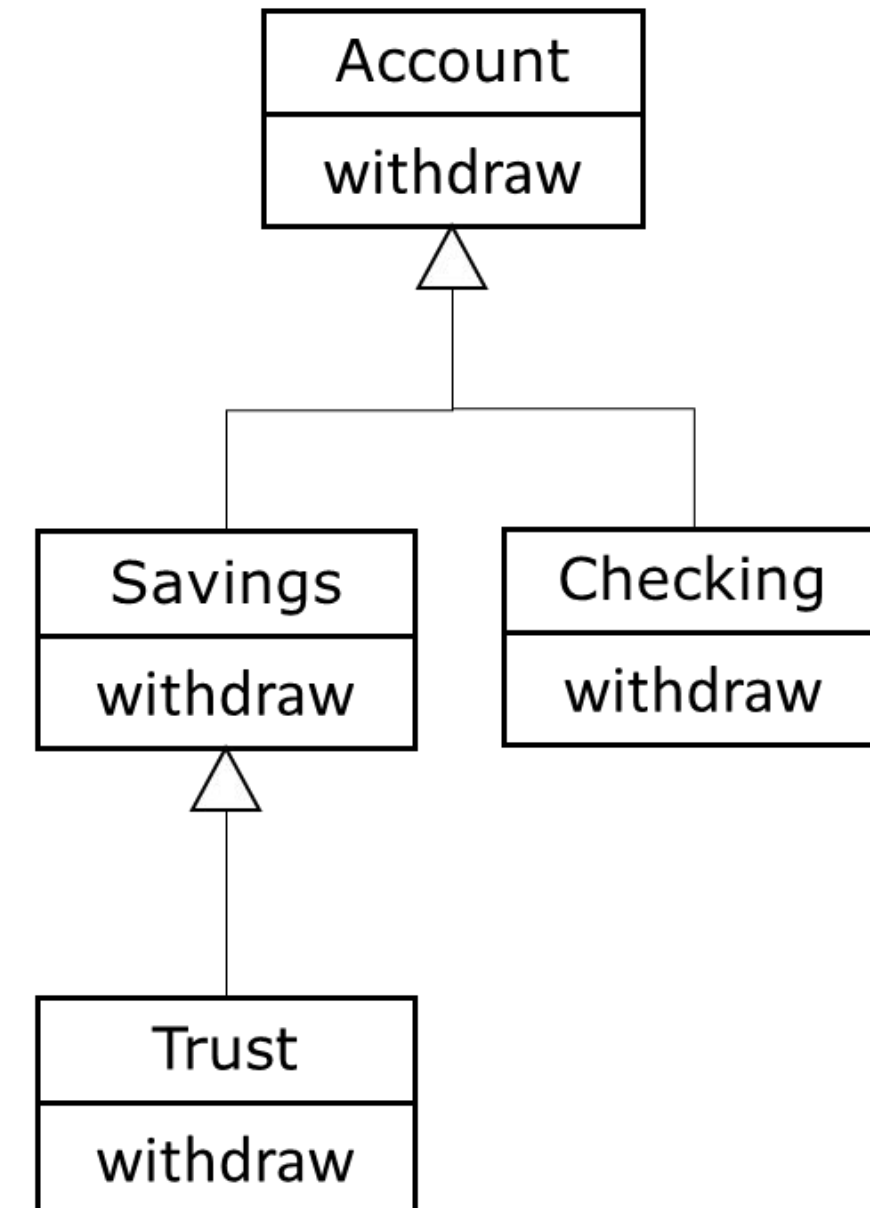
Types of Polymorphism in C++?



Polymorphism

An non-polymorphic example – Static Binding

```
Account a;  
a.withdraw(1000);    // Account::withdraw()  
  
Savings b;  
b.withdraw(1000);    // Savings::withdraw()  
  
Checking c;  
c.withdraw(1000);    // Checking::withdraw()  
  
Trust d;  
d.withdraw(1000);    // Trust::withdraw()  
  
Account *p = new Trust();  
P->withdraw(1000);    // Account::withdraw()  
                        // should be  
                        // Trust::withdraw()
```



Polymorphism

An non-polymorphic example – Static Binding

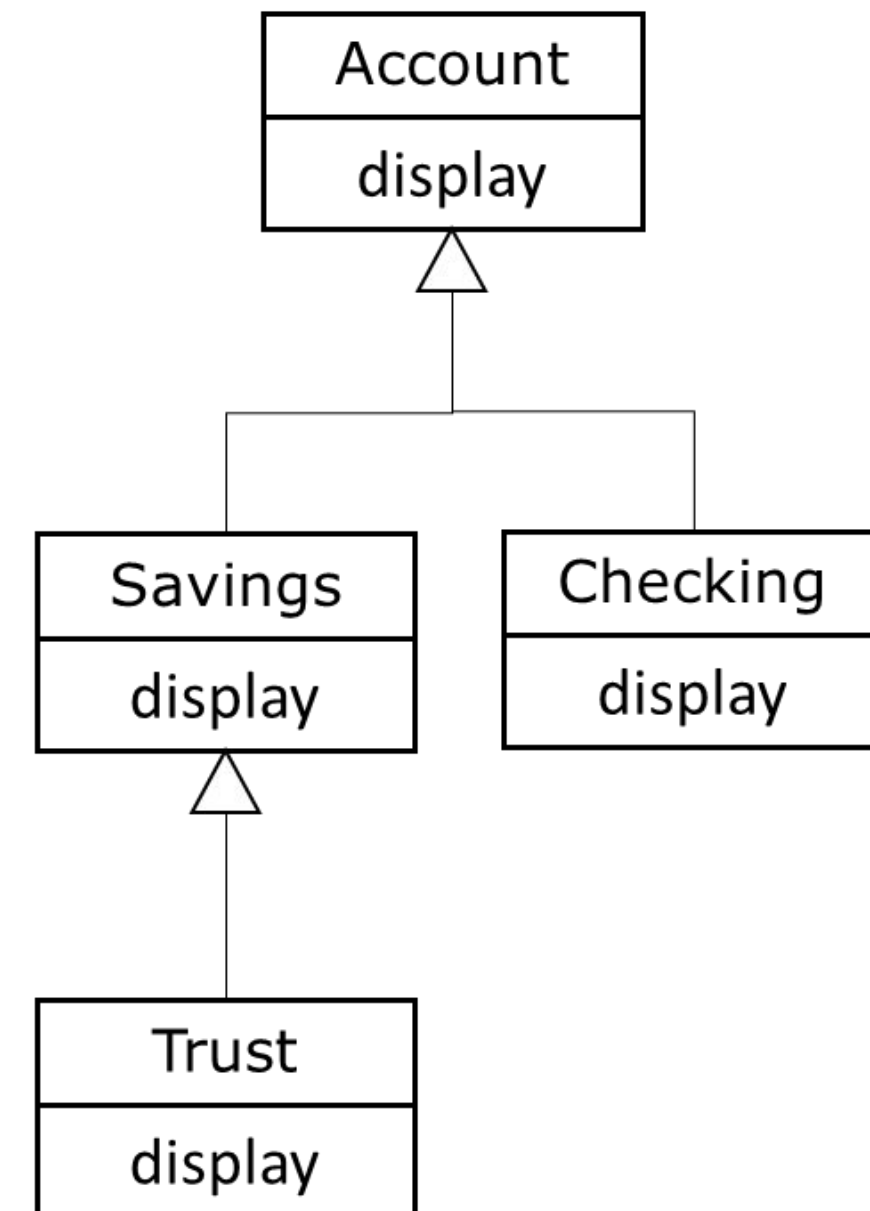
```
void display_account(const Account &acc) {  
    acc.display() ;  
    // will always use Account::display  
}
```

```
Account a;  
display_account(a);
```

```
Savings b;  
display_account(b);
```

```
Checking c;  
display_account(c);
```

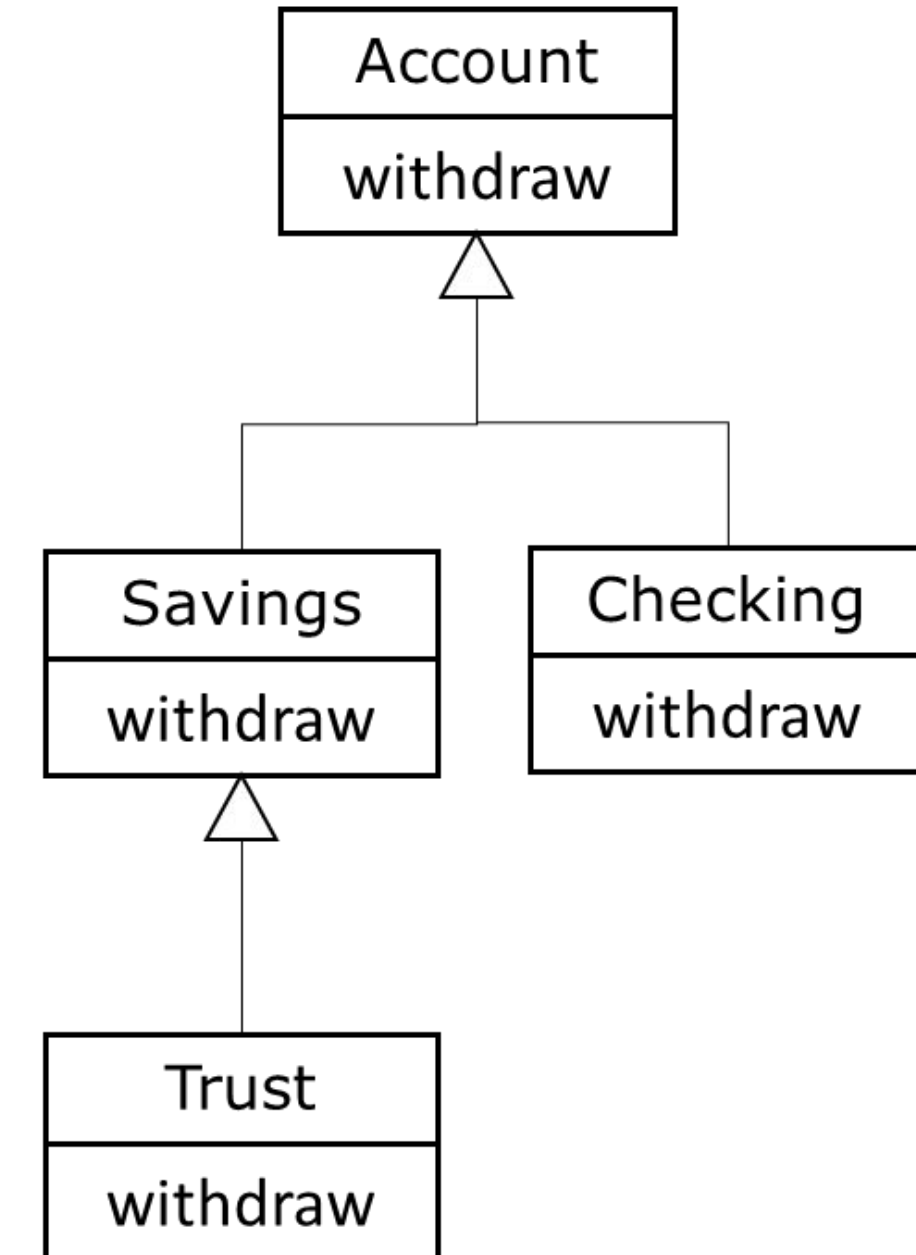
```
Trust d;  
display_account(d);
```



Polymorphism

A polymorphic example – Dynamic Binding

```
Account a;  
a.withdraw(1000);    // Account::withdraw()  
  
Savings b;  
b.withdraw(1000);    // Savings::withdraw()  
  
Checking c;  
c.withdraw(1000);    // Checking::withdraw()  
  
Trust d;  
d.withdraw(1000);    // Trust::withdraw()  
  
Account *p = new Trust();  
p->withdraw(1000);    // Trust::withdraw()
```



withdraw method is virtual in Account

Polymorphism

A polymorphic example – Dynamic Binding

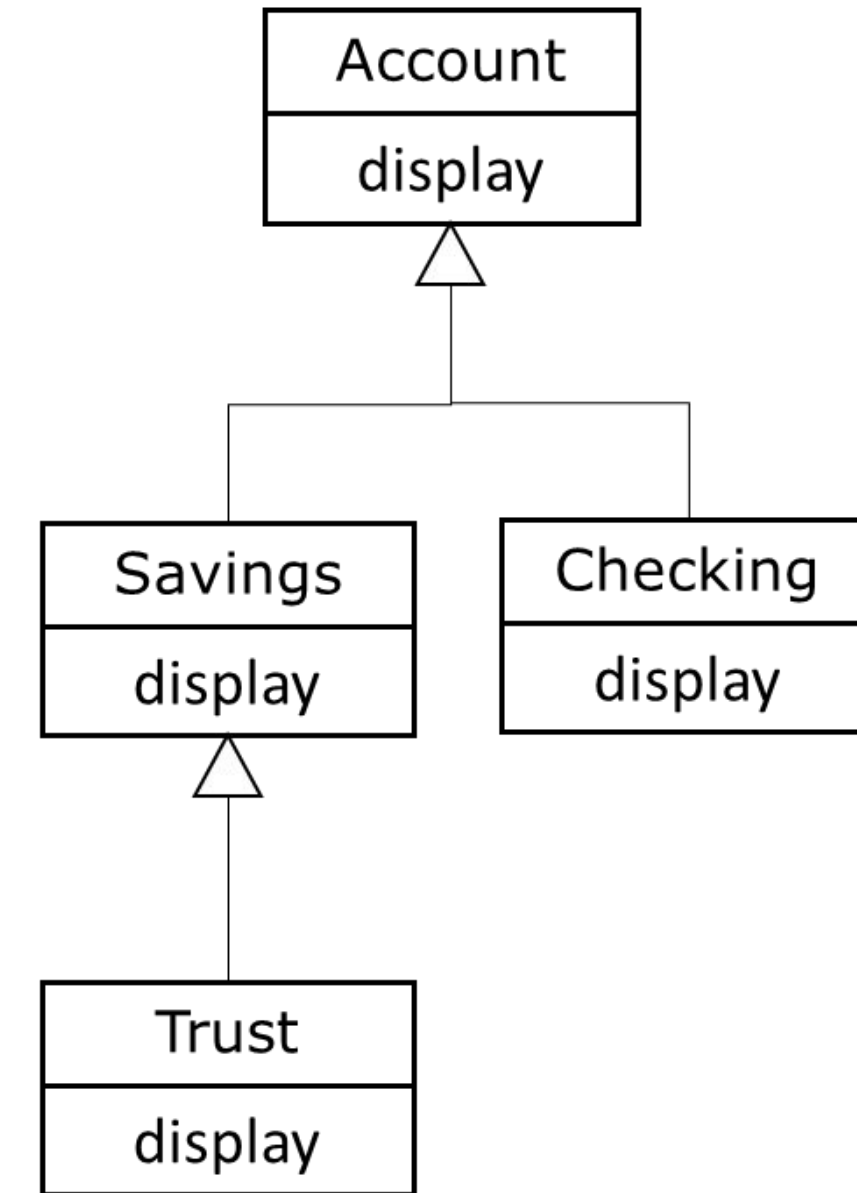
```
void display_account(const Account &acc)
{
    acc.display() ;
    // will always call the display method
    // depending on the object's type
    // at RUN-TIME!
}
```

```
Account a;
display_account(a);
```

```
Savings b;
display_account(b);
```

```
Checking c;
display_account(c);
```

```
Trust d;
display_account(d);
```



display method is virtual in Account

Polymorphism

Using a Base class pointer

- For dynamic polymorphism we must have:
 - Inheritance
 - Base class pointer or Base class reference
 - virtual functions

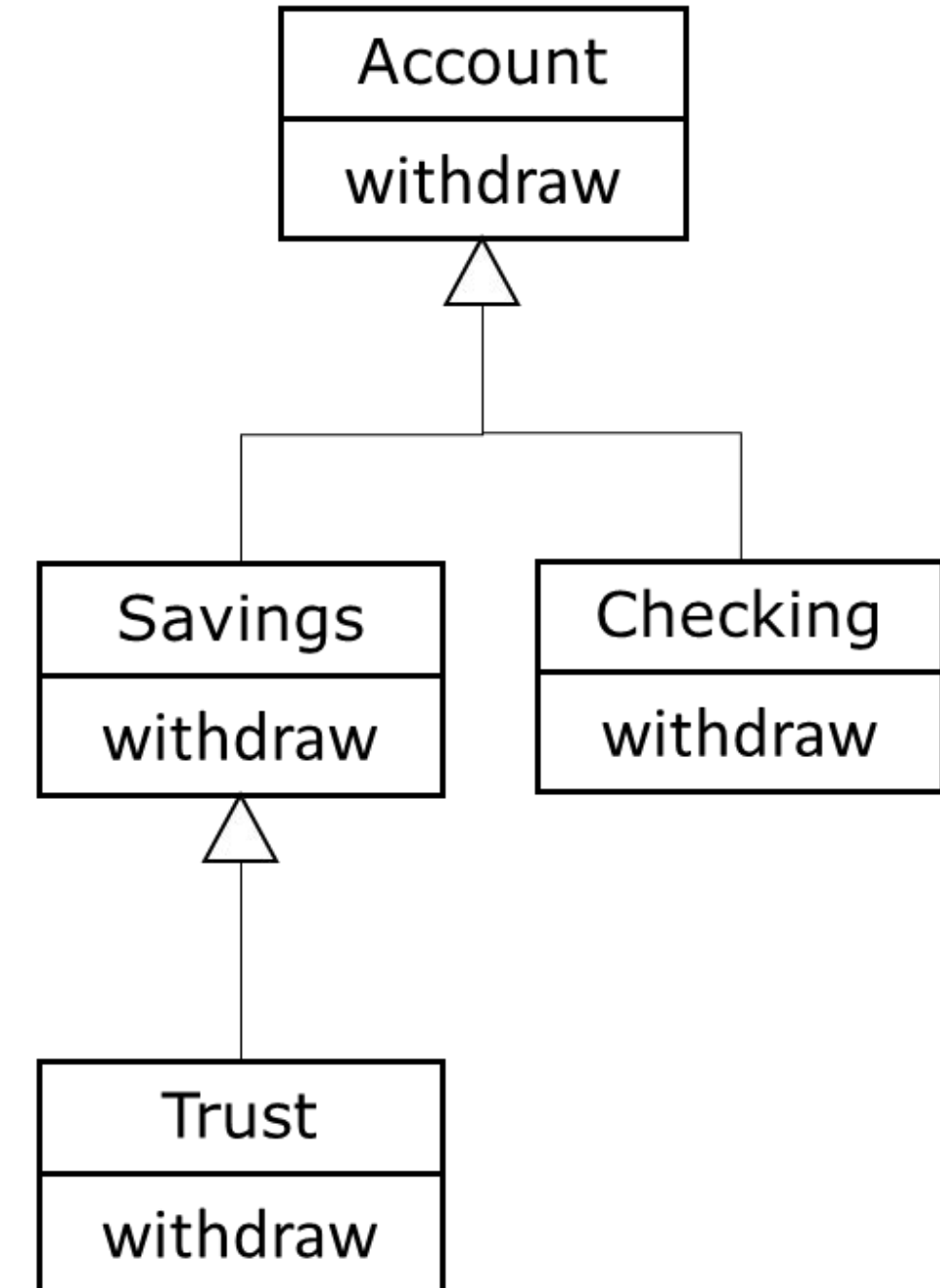
Polymorphism

Using a Base class pointer

```
Account *p1 = new Account();  
Account *p2 = new Savings();  
Account *p3 = new Checking();  
Account *p4 = new Trust();
```

```
p1->withdraw(1000); // Account::withdraw  
p2->withdraw(1000); // Savings::withdraw  
p3->withdraw(1000); // Checking::withdraw  
p4->withdraw(1000); // Trust::withdraw
```

```
// delete the pointers
```



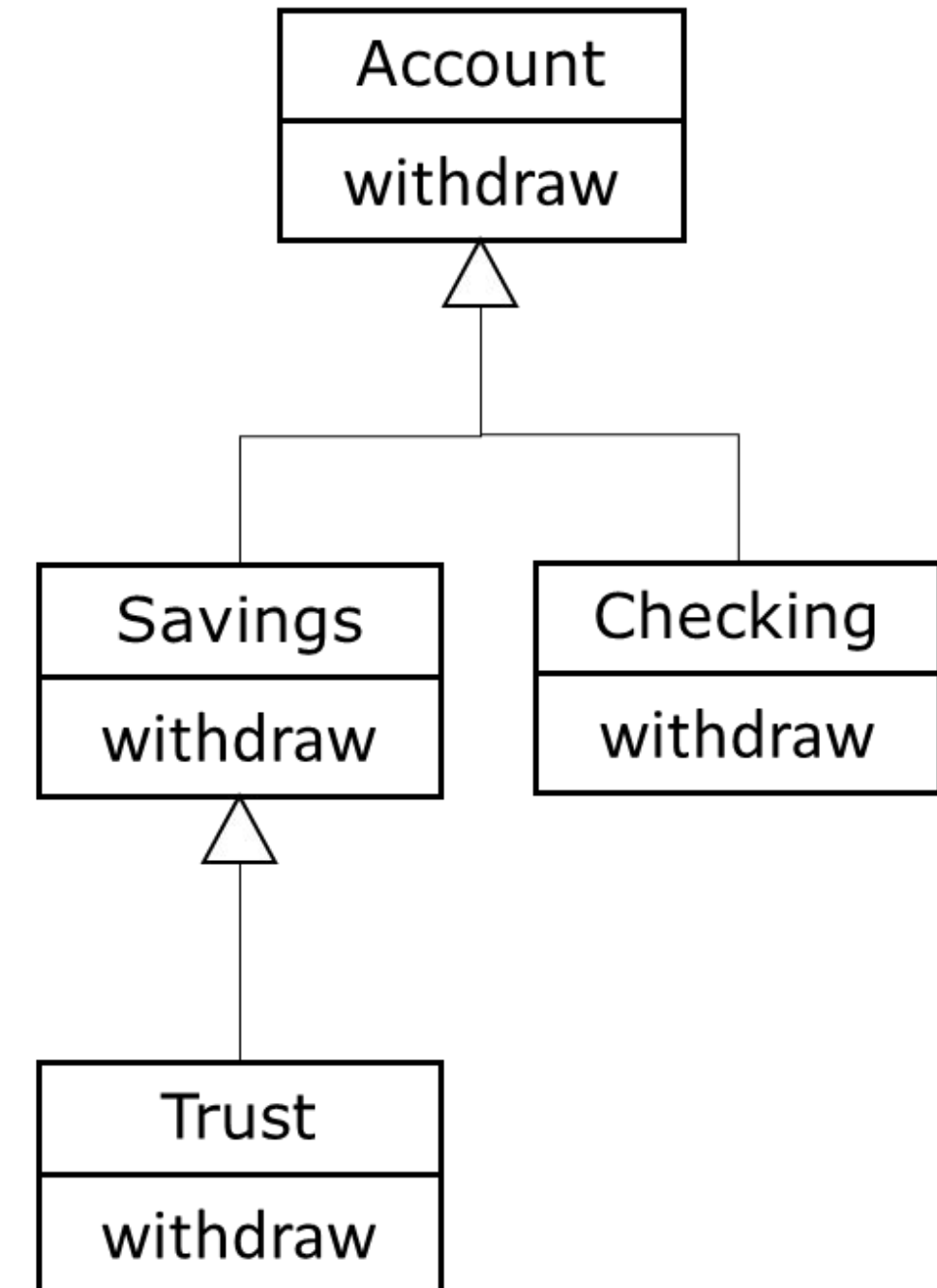
Polymorphism

Using a Base class pointer

```
Account *p1 = new Account();  
Account *p2 = new Savings();  
Account *p3 = new Checking();  
Account *p4 = new Trust();
```

```
Account *array [] = {p1,p2,p3,p4};
```

```
for (auto i=0; i<4; ++i)  
    array[i]->withdraw(1000);
```



Polymorphism

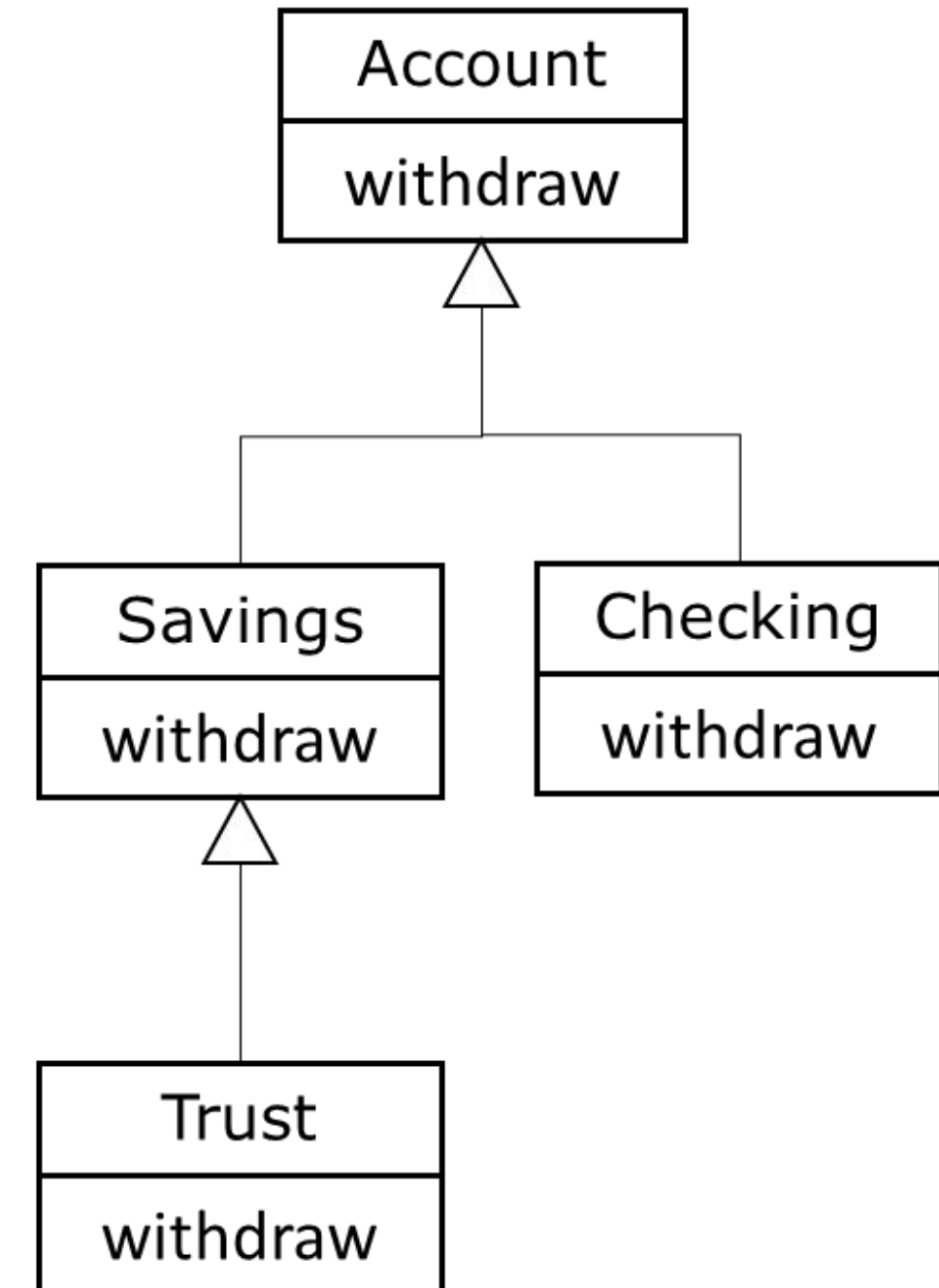
Using a Base class pointer

```
Account *p1 = new Account();  
Account *p2 = new Savings();  
Account *p3 = new Checking();  
Account *p4 = new Trust();
```

```
vector<Account *> accounts  
    {p1, p2, p3, p4};
```

```
for (auto acc_ptr: accounts)  
    acc_ptr->withdraw(1000);
```

```
// delete the pointers
```



Polymorphism

Virtual functions

- Redefined functions are bound statically
- Overridden functions are bound dynamically
- Virtual functions are overridden
- Allow us to treat all objects generally as objects of the Base class

Polymorphism

Declaring virtual functions

- Declare the function you want to override as virtual in the Base class
- Virtual functions are virtual all the way down the hierarchy from this point
- Dynamic polymorphism only via Account class pointer or reference

```
class Account {  
public:  
    virtual void withdraw(double amount);  
    . . .  
};
```


Polymorphism

Declaring virtual functions

- Override the function in the Derived classes
- Function signature and return type must match EXACTLY
- Virtual keyword not required but is best practice
- If you don't provide an overridden version it is inherited from it's base class

```
class Checking : public Account {  
public:  
    virtual void withdraw(double amount);  
    . . .  
};
```

Polymorphism

Virtual Destructors

- Problems can happen when we destroy polymorphic objects
- If a derived class is destroyed by deleting its storage via the base class pointer and the class has a non-virtual destructor. Then the behavior is undefined in the C++ standard.
- Derived objects must be destroyed in the correct order starting at the correct destructor

Polymorphism

Virtual Destructors

- Solution/Rule:
 - If a class has virtual functions
 - ALWAYS provide a public virtual destructor
 - If base class destructor is virtual then all derived class destructors are also virtual

```
class Account {  
public:  
    virtual void withdraw(double amount);  
    virtual ~Account();  
    . . .  
};
```

Polymorphism

The override specifier

- We can override Base class virtual functions
- The function signature and return must be EXACTLY the same
- If they are different then we have redefinition NOT overriding
- Redefinition is statically bound
- Overriding is dynamically bound
- C++11 provides an override specifier to have the compiler ensure overriding

Polymorphism

The override specifier

```
class Base {
public:
    virtual void say_hello() const {
        std::cout << "Hello - I'm a Base class object" << std::endl;
    }
    virtual ~Base() {}
};

class Derived: public Base {
public:
    virtual void say_hello() { // Notice I forgot the const - NOT OVERRIDING
        std::cout << "Hello - I'm a Derived class object" << std::endl;
    }
    virtual ~Derived() {}
};
```


Polymorphism

The override specifier

Base:

```
virtual void say_hello() const;
```

Derived:

```
virtual void say_hello();
```

Polymorphism

The override specifier

```
Base *p1 = new Base();  
p1->say_hello();           // "Hello - I'm a Base class object"
```

```
Base *p2 = new Derived();  
p2->say_hello();           // "Hello - I'm a Base class object"
```

- Not what we expected
- say_hello method signatures are different
- So Derived **redefines** say_hello instead of overriding it!

Polymorphism

The override specifier

```
class Base {
public:
    virtual void say_hello() const {
        std::cout << "Hello - I'm a Base class object" << std::endl;
    }
    virtual ~Base() {}
};

class Derived: public Base {
public:
    virtual void say_hello() override {    // Produces compiler error
                                           // Error: marked override but does not override

        std::cout << "Hello - I'm a Derived class object" << std::endl;
    }
    virtual ~Derived() {}
};
```


Polymorphism

The `final` specifier

- C++11 provides the final specifier
 - When used at the class level
 - Prevents a class from being derived from
 - When used at the method level
 - Prevents virtual method from being overridden in derived classes

Polymorphism

`final` class

```
class My_class final {  
    . . .  
};
```

```
class Derived final: public Base {  
    . . .  
};
```

Polymorphism

final method

```
class A {  
public:  
    virtual void do_something();  
};  
  
class B: public A {  
    virtual void do_something() final;    // prevent futher overriding  
};  
  
class C: public B {  
    virtual void do_something();           // COMPILER ERROR - Can't override  
};
```

Polymorphism

Using Base class references

- We can also use Base class references with dynamic polymorphism
- Useful when we pass objects to functions that expect a Base class reference

Polymorphism

Using Base class references

```
Account a;  
Account &ref = a;  
ref.withdraw(1000); // Account::withdraw
```

```
Trust t;  
Account &ref1 = t;  
ref1.withdraw(1000); // Trust::withdraw
```


Polymorphism

Using Base class references

```
void do_withdraw(Account &account, double amount) {  
    account.withdraw(amount);  
}
```

```
Account a;
```

```
do_withdraw(a, 1000); // Account::withdraw
```

```
Trust t;
```

```
do_withdraw(t, 1000); // Trust::withdraw
```

Polymorphism

Pure virtual functions and abstract classes

- Abstract class
 - Cannot instantiate objects
 - These classes are used as base classes in inheritance hierarchies
 - Often referred to as Abstract Base Classes
- Concrete class
 - Used to instantiate objects from
 - All their member functions are defined
 - Checking Account, Savings Account
 - Faculty, Staff
 - Enemy, Level Boss

Polymorphism

Pure virtual functions and abstract classes

- Abstract base class
 - Too generic to create objects from
 - Shape, Employee, Account, Player
 - Serves as parent to Derived classes that may have objects
 - Contains at least one pure virtual function